

Mossa 37

Documentazione Tecnica

© 2025-2026 Grafica Nappa S.r.l. — Tutti i diritti riservati.

Documento riservato. Distribuzione limitata ai partner autorizzati.

Mossa 37 — Documentazione Tecnica

© 2025-2026 Grafica Nappa S.r.l. — Tutti i diritti riservati. Documento riservato.

Distribuzione limitata ai partner autorizzati.

Indice

1. [Executive Summary](#) — cosa fa Mossa 37 e perché esiste
 2. [Architettura](#) — Laravel 12 + DDD modulare
 3. [Stack Tecnico](#) — PHP 8.5, MySQL, Apache, Windows Server
 4. [Moduli Funzionali](#) — Onda, Operatori, Spedizione, Prinect, Scheduling
 5. [Integrazioni](#) — Onda SQL Server, Prinect REST, BRT, NetTime, Solar-Log, Telegram
 6. [Database](#) — schema 115 tabelle, ER principale
 7. [Flussi](#) — sequence diagrams: sync Onda, ciclo fase, DDT
 8. [Deployment](#) — Windows Server, cron Task Scheduler, queue worker
 9. [Sicurezza e GDPR](#) — 2FA, ruoli, audit log, art. 4 Statuto Lavoratori
-

Panoramica rapida

Voce	Valore
Linguaggio	PHP 8.5
Framework	Laravel 12
Database operativo	MySQL 8
Database ERP sorgente	SQL Server (Onda)
Architettura	DDD modulare (19 moduli)
Moduli app/Modules/	Onda, Operatori, Spedizione, Prinect, Magazzino, Scheduling, Fasi, Carta, Fustelle, Macchine, Notifiche, Presenze, Reparti, Reportistica, Stampa, Commessa, Documenti, Audit
Migrations DB	115
Rotte web	427 righe routes/web.php
Tipo deploy	On-premise Windows Server + Apache
Cron	Windows Task Scheduler (schedule:run ogni minuto)
Queue	Database driver + worker batch ogni 2 min
Sync ERP	Onda SQL Server ogni ora
Sync Prinect (stampa)	REST API ogni minuto
Sync Fiery (digitale)	Accounting API ogni minuto
Excel bidirezionale	Ogni 2 minuti

Cosa fa Mossa 37

Sistema di esecuzione manifatturiera per **tipografia industriale (stampa offset + digitale + finitura + legatoria + spedizione)**.

Funzioni principali:

- **Tracciamento commesse:** dal recepimento ordine cliente (via ERP Onda) alla spedizione DDT

- **Stato fasi real-time:** ogni operatore registra avvio/pausa/termine fase su dashboard touch
- **Sincronizzazione macchine:** Prinect (stampa offset XL106) + Fiery (digitale) inviano fogli prodotti, scarti, tempi automatici
- **Schedulazione produzione:** scheduler con propagazione fasi e ottimizzazione setup batch
- **Spedizione:** gestione DDT BRT + corriere proprio, etichette Data Matrix
- **Reportistica:** ore lavorate vs preventivate, KPI per reparto, marginalità commessa
- **Magazzino carta:** giacenze, movimentazione, allerte sotto-scorta
- **Esterno:** gestione lavorazioni esterne (fustellatura, plastificazione, finitura)
- **Prestampa:** gestione cliché, fustelle, note pre stampa, operatore pre stampa

Target utenti

Ruolo	Dashboard	Device
Direzione	/owner	PC desktop
Operatore reparto	/operatore	Tablet touch + smartphone
Spedizione	/spedizione	PC + lettore Data Matrix
Prestampa	/operatore/prestampa	PC
Admin IT	/admin	PC
Fiery operator	/fiery	PC

01. Executive Summary

Contesto industriale

è una tipografia industriale specializzata in **astucci e packaging**, con linea produttiva integrata:

- **Stampa offset** (Heidelberg Speedmaster XL106 24h/24, 5 macchine)
- **Stampa digitale** (Fiery + HP Indigo)
- **Finitura digitale** (Zund taglio, MGI stampa lamina/oro a caldo)
- **Plastificazione** (lucida, opaca, soft-touch)
- **Stampa a caldo JOH** (lamina oro/argento)
- **UV spot vernice**
- **Fustellatura** (Bobst per rilievi e fustelle piane)
- **Piegaincolla** (3 configurazioni)
- **Legatoria** (brossura filo refe, brossura fresata, punto metallico, arrotondamento)
- **Spedizione** (BRT corriere principale + corriere proprio)

Problema risolto

Prima di Mossa 37 la produzione era gestita con:

- **ERP Onda (SQL Server)** per il commerciale (offerte, ordini, fatture, magazzino)
- **Excel** distribuiti via rete per pianificazione produzione e stato avanzamento
- **Telefono** per comunicazione tra reparti
- **Stampe cartacee** per schede produzione e DDT manuali

Conseguenze:

- Nessuna visibilità real-time dello stato fasi
- Doppia digitazione (ERP → Excel → reparti)
- Scarti e tempi produttivi non tracciati
- Difficoltà nel calcolo della marginalità per commessa
- Ritardi nella spedizione per mancata sincronizzazione finitura ↔ spedizione

Soluzione MES

Il Mossa 37 è una piattaforma web on-premise che:

1. **Sincronizza automaticamente** ordini, fasi, articoli e materiali da Onda ERP (ogni ora)
2. **Sincronizza automaticamente** dati produzione da Prinect (stampa offset) e Fiery (digitale) ogni minuto
3. **Fornisce dashboard ruolo-specifiche** per owner, operatori, spedizione, pre stampa
4. **Permette tracciamento real-time** dello stato fasi (avvio, pausa con motivazione, termine, qta prodotta, scarti)
5. **Genera etichette Data Matrix** per ogni commessa con BarCode reader-friendly
6. **Calcola KPI** ore lavorate vs preventivate, scarti reali vs previsti, marginalità
7. **Notifica** automaticamente la spedizione quando una commessa è pronta o inviata in lavorazione esterna
8. **Esporta DDT** verso BRT (XML SOAP) e produce DDT corriere proprio (PDF)
9. **Schedula produzione** con scheduler Mossa 37 (propagazione fasi + ottimizzazione setup macchine)

Risultati misurabili (dopo 12 mesi di adozione)

KPI	Valore
Tempo medio passaggio fase → reparto successivo	-45%
Errori di digitazione manuale ordini	-90%
Visibilità stato commessa per direzione	da daily snapshot → real-time
Tempo redazione DDT spedizione	-70% (auto-generazione da fasi terminate)
Tracciabilità ore lavorate per commessa	da stima → dato puntuale Prinect-derivato

Posizionamento

Mossa 37 si posiziona come **complemento all'ERP**, non sostituto:

- **Onda ERP**: rimane sorgente di verità per anagrafiche, listini, fatturazione, magazzino contabile, OC cliente
- **MES**: sorgente di verità per stato produzione, scarti reali, tempi reali, etichette, DDT trasporto

Roadmap futura prevede la **sostituzione progressiva di Onda** quando Mossa 37 avrà coperto tutte le funzionalità (Q4 2027+, vedi [10-roadmap.md](#)).

Architettura sintetica



02. Architettura

Pattern: monolite modulare DDD-lite

L'app è un **monolite Laravel 12** organizzato in **moduli DDD** in `app/Modules/`, seguendo il pattern **architettura DDD modulare** per la migrazione del codice legacy.

architettura DDD modulare = il codice nuovo "avvolge" e gradualmente sostituisce il legacy senza big-bang refactoring. Il sistema attuale ha completato la migrazione dei moduli core (Onda, Spedizione, Commessa, Reportistica, Audit).

Diagramma layer

flowchart TB

```
Browser["Browser / Tablet / Mobile"] -->|HTTPS| Apache["Apache 2.4 + PHP FastCGI"]
Telegram["Bot Telegram"] -->|webhook| Apache
Scheduler["Windows Task Scheduler"] -->|artisan| Console["Artisan Commands"]
Apache --> Controllers["app/Http/Controllers<br/>(thin: 36 controller)"]
Console --> Modules
Controllers --> Modules["app/Modules/<br/>(19 moduli DDD)"]
Modules --> Eloquent["app/Models<br/>(Eloquent ORM)"]
Modules --> Adapters["Modules/*/Adapters<br/>(anti-corruption)"]
Adapters --> Legacy["app/Services<br/>(legacy adapter)"]
Adapters --> External["Integrazioni esterne"]
Eloquent --> MySQL["MySQL (MySQL moss37)"]
External --> Onda["SQL Server Onda"]
External --> Prinect["Prinect REST"]
External --> BRT["BRT SOAP"]
External --> Fiery["Fiery REST"]
External --> NetTime["NetTime CSV share"]
```

Direzione dipendenze

`Controller → Modules → {Eloquent, Adapter → Legacy/External}`

Vincoli:

- I moduli **non chiamano direttamente** HTTP/SOAP esterni → passano da `Contracts` + `Adapters` (anti-corruption layer)
- Eloquent è permesso nei moduli (pragmatismo, no repository pattern forzato finché lo schema MySQL resta lo stesso)
- I Controller sono **thin**: parse request, auth/CSRF, delega al modulo, formatta risposta
- DI via container Laravel — `new` diretto vietato in business logic

Struttura modulo tipo

```
app/Modules/<Nome>/
├─ Contracts/      # Interfacce (porte verso external/legacy)
├─ Adapters/       # Implementazioni concrete (anti-corruption)
├─ Services/       # Use case (un metodo pubblico = un caso d'uso)
├─ Rules/          # Logica pura, testabile in isolamento
├─ Enums/          # PHP 8.1 backed enum (string|int)
├─ Events/         # DTO readonly Dispatchable
├─ Listeners/      # Reazioni cross-modulo
├─ ValueObjects/   # Readonly, immutabili, factory ::from()
├─ Exceptions/     # Eccezioni di dominio
└─ StateMachine/  # Opzionale (es. Fasi)
```

Convenzioni codice

- **PHP 8.5** strict types, readonly properties, enum backed
- **Laravel 12** (PHP minimum 8.2 da composer.json, prod 8.5)
- **PHPStan level 5** + Larastan (CI bloccante)
- **Laravel Pint** (PSR-12)
- Classi `final` di default, ereditarietà solo se necessaria
- Dipendenze iniettate via container, no `new` diretto in business logic
- Eccezioni di dominio in `Exceptions/`, no `\Exception` generica

I 19 moduli (alfabetico)

Modulo	Responsabilità sintetica
Audit	Logging eventi business + compliance GDPR/Statuto art. 4
Carta	Anagrafica carta (prefisso 02W. Onda), parser codice, conversione fogli↔kg
Commessa	Aggregato commessa, stato derivato, qta totale/consegnata
Documenti	Generazione etichette DataMatrix, Excel bidirezionale
Fasi	State machine OrdineFase (transizioni 0-5, pause con motivo)
Fustelle	Anagrafica fustelle tipizzata (separata da legacy <code>cliche_anagrafica</code>)
Macchine	Registry 12 macchine fisiche + regole turni/capacità/setup
Magazzino	Orchestratore movimenti carta (carico/scarico/reso/rettifica)
Notifiche	Fan-out multi-canale routing per priorità (Push/Email/Telegram)
Onda	Integrazione ERP Onda (SQL Server) → MES
Operatori	Matrice ruolo→permessi, controllo autorizzazione MES
Presenze	NetTime sync, calcolo ore, assenze (art. 4 Statuto)
Prinect	Integrazione Heidelberg Prinect XL106 (REST)
Reparti	Tipizzazione reparti, capacità/turni per scheduler
Reportistica	Aggregazioni KPI, report ore, panoramiche, marginalità
Scheduling	Scheduler Mossa 37 — priorità 4 livelli + propagazione cascata
Spedizione	Tracking BRT, DDT MES, note consegne bidirezionali
Stampa	Astrazione comune Prinect (offset) + Fiery (digitale)

Nota: **non esiste** un modulo `Owner` separato — la logica owner è distribuita in Commessa, Fasi, Scheduling, Spedizione, Reportistica. Il controller `DashboardOwnerController` orchestra i moduli.

Eventi cross-modulo

Dispatchati e gestiti via Laravel Events:

Evento	Dispatcher	Listener
FaseAvviata	Fasi	TracciaInizioFase (Audit), aggiorna pivot operatore
FaseTerminata	Fasi	PropagaFasiSuccessive (Scheduling), NotificaCommessaCompletata
CommessaCompletata	Commessa	Notifica spedizione + audit
OrdineSincronizzato	Onda	Excel sync trigger
WorkstepCompleted	Prinect	Update fase MES, calc tempi
SottoSogliaEvento	Magazzino	NotificaSottoSoglia
SpedizioneInRitardo	Spedizione	NotificaSpedizioneInRitardo
FustellaPrelevata / Restituita	Fustelle	Audit + lock check
TimbraturaRegistrata	Presenze	Calcolo straordinari

Test strategy

- **Unit:** Rules/ , ValueObjects/ , Services/ con mock di Contracts (PHPUnit/Pest)
- **Integration:** Service via container DI, DB transactional rollback
- **Feature:** HTTP testing via Laravel TestCase
- **Static analysis:** PHPStan level 5 bloccante in CI (larastan ^3.0)
- **Style:** Laravel Pint PSR-12 in CI

Documenti interni

Documentazione architetturale completa nel repo:

Razionale architetturale

Quattro obiettivi del refactoring DDD:

1. **Isolare il dominio** dai concern HTTP/persistenza Eloquent — la logica business si testa senza HTTP e senza DB Eloquent
2. **Testabilità** — classi `final` , `readonly` , strict types, dipendenze iniettate
3. **Migrazione incrementale** — architettura DDD modulare: nessun big-bang. I moduli avvolgono i service legacy tramite Adapter ed espongono dietro Contracts.

Cambio implementazione = swap di un Adapter, business logic intatta.

4. **Riuso futuro come SaaS multi-tenant** — la separazione dominio/HTTP rende possibile (a tendere) hostare istanze isolate dello stesso codebase per più tipografie. Il sistema attuale predispone questo scenario.

03. Stack Tecnico

Sintesi

Componente	Versione	Ruolo
PHP	8.5 (min 8.2 da composer)	Runtime backend
Laravel	12	Framework
MySQL	8	DB operativo MES
SQL Server	(Onda ERP)	Sorgente dati ERP
Apache	2.4	Web server (FastCGI PHP)
Vite	7	Bundler frontend
Bootstrap	5.2+	UI framework
Tailwind	4	Utility CSS

Backend

Linguaggio + Framework

- **PHP 8.5** in produzione (min 8.2 dichiarato in `composer.json`)
- **Laravel 12** — framework MVC
- Strict types, readonly properties, enum backed (PHP 8.1+ features)

Dipendenze Composer (`require`)

Pacchetto	Versione	Scopo
laravel/framework	^12.0	Core framework
laravel/reverb	^1.8	WebSocket server (real-time, opzionale)
laravel/tinker	^2.10	REPL CLI
laravel/ui	^4.6	Scaffolding UI auth
barryvdh/laravel-dompdf	^3.1	Generazione PDF (DDT, etichette, schede produzione)
maatwebsite/excel	^3.1	Import/export Excel (PhpSpreadsheet)
simplesoftwareio/simple-qr-code	^4.2	QR e DataMatrix etichette
spatie/laravel-permission	^7.4	RBAC (ruoli/permessi)
thiagoaleissio/tesseract_ocr	^2.13	OCR bolle magazzino (foto Telegram)
pusher/pusher-php-server	^7.2	Push notifications (alternativa a Reverb)

Dipendenze on-demand (non in composer.json corrente, da verificare branch attivo)

Pacchetto	Scopo
pragmarx/google2fa	TOTP 2FA admin
tecnickcom/tcpdf	PDF avanzati (alternativa)
irazasyed/telegram-bot-sdk	SDK bot Telegram
minishlink/web-push	Web Push notifications
phpoffice/phpspreadsheet	Manipolazione Excel (transitive da maatwebsite/excel)

Frontend

Build tooling

- **Vite 7** — bundler (config vite.config.js)
- **laravel-vite-plugin** ^2.0
- **concurrently** ^9 — orchestrazione dev processes

- **sass** ^1.56 — preprocessor

Libraries UI

Libreria	Versione	Uso
Blade templates	(Laravel)	Server-side templating
Bootstrap	^5.2	UI framework
@popperjs/core	^2.11	Tooltip/popover positioning
Tailwind CSS	^4.0	Utility-first CSS
@tailwindcss/vite	^4.0	Integrazione Vite
axios	^1.11	HTTP client AJAX
Choices.js	(CDN)	Multi-select filtering
Handsontable	v14 (CDN)	Excel-like bulk edit (modifica multipla owner)
Html5Qrcode	(CDN lazy)	QR scanner etichette
Chart.js	(CDN)	KPI visualizations (report ore, dashboard owner)

PWA

- Manifest `public/manifest.json`
- Service worker per cache offline
- Push notifications via Web Push API (VAPID keys)
- Installabile su tablet operatori (home screen icon)

WebSocket (opzionale)

- **Laravel Reverb** + **Laravel Echo** per chat real-time
- In produzione attualmente NON attivo (polling 15s come fallback)

Storage

Tipo	Path	Uso
Filesystem locale	<code>storage/app/</code>	PDF DDT, etichette, log applicativi
Filesystem locale	<code>storage/app/excel_sync/</code>	File Excel bidirezionale (<code>sync_data.xlsx</code>)
Filesystem locale	<code>public/</code>	Asset statici, immagini, manifest
Share di rete	<code>\\.\253\nettime_timbrature\</code>	CSV timbrature NetTime
Share di rete	<code>\\.\34\</code>	Export NetTime PC (TODO config)

Deploy

Componente	Configurazione
OS server	Windows Server (Server cliente)
Web server	Apache 2.4 con PHP FastCGI
HTTPS	Forced via env <code>force_https</code> (default off, on in prod)
Web app	Apache su porta 80, single deployment
DB	MySQL 8 locale, connessione TCP 3306
Cron	Windows Task Scheduler → <code>php artisan schedule:run</code> ogni minuto
Queue	Driver <code>database</code> , worker <code>queue_worker.bat</code> invocato ogni 2 min
Servizi Windows	<code>Mossa37TelegramBot</code> (nssm) per long-polling bot

Dettaglio completo in `08-deployment.md` .

Requisiti minimi server

Produzione (carico : ~30 utenti concorrenti, 6000 ordini storici)

Risorsa	Minimo	Raccomandato
CPU	4 core	8 core
RAM	8 GB	16 GB
Disco	100 GB SSD	250 GB SSD
OS	Windows Server 2019+	Windows Server 2022
PHP	8.2	8.5
MySQL	8.0	8.0 LTS
Apache	2.4	2.4

Sviluppo locale

Risorsa	Minimo
CPU	4 core
RAM	8 GB
Disco	30 GB
OS	Windows 10/11, macOS, Linux
PHP	8.2+
MySQL	8.0
Node	18+ (per vite)
Composer	2.5+

Configurazione SSL/TLS

- **HTTPS**: certificato self-signed o Let's Encrypt
- **HSTS**: max-age 1 anno (attivato solo dopo verifica HTTPS stabile)
- **CSP**: Report-Only attualmente (legacy unsafe-inline/unsafe-eval per onclick/Echo)

Lingua / Localizzazione

- **UI**: italiano
- **Codice**: italiano (nomi metodi, classi, commenti)

- **Database:** italiano (nomi colonne, tabelle)
- **Date:** formato italiano (`d/m/Y` , `d/m/Y H:i:s`)
- **Decimali:** separatore `,` (visualizzazione), `.` (storage)

04. Moduli Funzionali

I 19 moduli DDD in `app/Modules/` orchestrano la business logic di Mossa 37. Ogni modulo è autonomo, ha contracts espliciti verso l'esterno e dipende da altri moduli solo via Eventi o Contracts.

Audit

Path: `app/Modules/Audit/` **Scopo:** Logging centralizzato di eventi business + compliance GDPR / Statuto Lavoratori art. 4.

- **Contracts:** `AuditSinkInterface`
- **Services:**
 - `AuditLogService::log()` — Registra evento auditato (azione, entità, prima/dopo) con sanitizzazione automatica dati sensibili
 - `AuditQueryService` — Read-side: ricerca log per utente/azione/modello
 - `ComplianceExportService::esportaPerOperatore()` / `::aggregatoSindacale()` — Export GDPR art. 15 + aggregato RSU pseudonimizzato
- **Adapters:** `DatabaseAuditSink`, `FileAuditSink`, `NullAuditSink`
- **Rules:** `DatiSensibiliRule` (mask password/token/Bearer/sk-*), `RitenzioneRule` (GDPR retention 3yr/2yr/7yr), `AccessoLogRule` (chi può leggere)
- **ValueObjects:** `AuditEvent`, `ContestoUtente`, `DiffPayload`
- **Enums:** `TipoAzione` (Create/Read/Update/Delete/Login/Logout/Export/Sync/Failed), `EntitaAudit`, `LivelloSicurezza` (Normale/Sensibile/Critico)
- **Listeners:** `LogFaseAvviata`, `LogFaseTerminata`, `LogCommessaCompletata`, `LogMovimentoMagazzino`, `LogLoginRiuscito`, `LogLoginFallito`

Carta

Path: `app/Modules/Carta/` **Scopo:** Anagrafica tipizzata carta (prefisso `02W.` Onda). Parser codice + conversione fogli↔kg + compatibilità macchina.

- **Services:** `AnagraficaCartaService` (`cerca`, `perFamiglia`, `ricerca`)

- **Rules:** `CompatibilitaMacchinaRule::isCompatibile()`, `ConversioneFogliKgRule` (`fogliToKg`, `kgToFogli`)
- **ValueObjects:** `CodiceArticoloOnda::from()` (parser regex 5 segmenti TIPO.NOME.QUALITA.GRAMMATURA.FORMATO), `Formato`
- **Enums:** `FamigliaCarta` (Patinata/Naturale/Cartone/Adesivo), `TipoSuperficie` (Lucida/Opaca/Satinata)

Commessa

Path: `app/Modules/Commessa/` **Scopo:** Vista aggregata (insieme Ordini stesso codice base): stato derivato, qta totale/consegnata, urgenza, dispatch eventi completamento.

- **Services:**
 - `CommessaService` (`getStatoAggregato`, `getQtaTotale`, `getQtaConsegnata`, `getOrdiniDellaCommessa`, `aggiornaCampo`)
 - `ProgressoService` (`percentuale`, `fasiResidue`)
- **Rules:** `StatoDerivatoRule::deriva()` (mapping stati fasi → StatoCommessa)
- **ValueObjects:** `CodiceCommessa::from()` (parser `NNNNNNN-AA`)
- **Enums:** `StatoCommessa` (Bozza/InCorso/Completata/Consegnata/Sospesa)
- **Events:** `CommessaCompletata`, `CommessaConsegnata`

Documenti

Path: `app/Modules/Documenti/` **Scopo:** Generazione documenti MES (Etichette DataMatrix con GTIN, schede produzione, DDT). Sync Excel bidirezionale Prinect.

- **Contracts:** `DocumentoGeneratorInterface`
- **Services:**
 - `DocumentoService::genera()` — Factory generazione con context
 - `EtichettaGenerator implements DocumentoGeneratorInterface` — DataMatrix A8 cifre zero-padded
 - `ExcelSyncService` (`syncIn`, `syncOut`) — `sync_data.xlsx` bidirezionale via queue
- **Rules:** `CodiceArticoloRule`, `GtinRule` (checksum)
- **ValueObjects:** `MetadatiDocumento` (codArt, articolo, lotto, qta, dataOrdinazione)

- **Enums:** `TipoDocumento` (Etichetta/SchedaProduzione/Bolla/DDT), `FormatoDocumento` (Pdf/Png/Xlsx)

Fasi

Path: `app/Modules/Fasi/` **Scopo:** State machine `OrdineFase`. Transizioni legali, pause con motivo, audit log strutturato, dispatch eventi dominio.

- **Services:** `FaseTransitionService` (`transizione` , `pausa` , `riprendi`)
- **Rules:** `PausaRule` (`canPausa` , `canRiprendi`), `TransizioneRule` (StateMachine registry)
- **Enums:** `StatoFase` — 0=NonIniziata, 1=Pronto, 2=Avviato, 3=Terminato, 4=Consegnato, 5=Esterna; pause = string motivo
- **Events:** `FaseAvviata(ordineFase, operatore, isRipresa)` , `FaseTerminata(ordineFase, operatore)`
- **Listeners:** `PropagaFasiSuccessiveListener` (cascata attivazione), `TracciaInizioFaseListener` , `NotificaCommessaCompletataListener`

Fustelle

Path: `app/Modules/Fustelle/` **Scopo:** Gestione tipizzata fustelle/cliché (anagrafica, stato magazzino, prelievi, note operative). Tabella nuova `fustelle` (distinta da legacy `cliche_anagrafica`).

- **Services:**
 - `FustellaService` (`cercaPerCodice` cache 1h, `crea` , `cambiaStato`)
 - `NoteFustellaService` (`aggiungi` append-only, `elenco`)
 - `PrelievoFustellaService` (`preleva` , `restituisce`)
- **Rules:** `FustellaCompatibileMacchina::eCompatibile()` , `ValidazioneCodiceFustella`
- **ValueObjects:**
 - `CodiceFustella::daStringa()` (`F-NNNNN-X`) + `::daLegacy()`
 - `DimensioneFustella::daStringa()` (`720x510x0.71`)
 - `NotaFustella::ora()` (timestamp `[Autore - dd/mm HH:MM]`)
- **Enums:** `TipoFustella` (PIANA/ROTATIVA/TRANCIATURA/RILIEVO, `::dalCodiceFase()` , `::macchineCompatibili()`), `StatoFustella` (PREPARAZIONE/PRONTA/IN_USO/ARCHIVIATA, `::puoTransitareA()`)

- **Events:** `FustellaPrelevata` , `FustellaRestituita` , `NotaFustellaAggiunta`

Macchine

Path: `app/Modules/Macchine/` **Scopo:** Registry in-memory 12 macchine fisiche + regole specifiche (turni, capacità, setup).

- **Contracts:** `MacchinaInterface` (`getId` , `getTurni` , `capacitaNominale`)
- **Services:** `CalcoloOreService::stima()`
- **Implementations:**
 - `RegoleXL106` — Heidelberg offset, 24h lun-ven
 - `RegoleJ0H` — Caldo, 6-22 lun-ven
 - `RegoleB0BST` — 1 macchina, 2 config (rilievi/fustelle), cambio 1h
 - `RegolePiegaincolla` — 1 macchina, 3 config (PI01/PI02/PI03), cambio 1h
 - `RegoleStandard` — 6-22: STEL, PLAST, FIN, INDIGO, TAGLIO, LEGAT, ZUND, MGI
- **Registry:** `MacchinaRegistry::all()` / `::find()` / `::exists()` (cache 1h)

Magazzino

Path: `app/Modules/Magazzino/` **Scopo:** Orchestratore magazzino carta: registra movimenti, aggiorna giacenza stessa transazione, annullamento via movimento opposto, evento sottosoglia.

- **Services:**
 - `MovimentoService` (`registraMovimento` con transaction, `annullaMovimento` opposto logico, `storicoMovimenti` 90gg)
 - `GiacenzaService` (`aggiornaGiacenza` delta + causale, `giacenzaCorrente`)
- **Rules:** `MovimentoValiditaRule` , `SogliaSottoMinimoRule` (emette evento)
- **ValueObjects:** `QuantitaCarta` (fogli o kg)
- **Enums:** `TipoMovimento` (Carico/Scarico/Reso/Rettifica), `StatoGiacenza` (InStock/SottoSoglia/Critica)
- **Events:** `SottoSogliaEvento(cod_art, giacenza_attuale, soglia)` → `NotificaSottoSogliaListener`

Notifiche

Path: `app/Modules/Notifiche/` **Scopo:** Orchestratore notifiche con fan-out multi-canale routing per priorità (Bassa→BrowserPush, Media→Email, Alta→Telegram, Critica→Tutti).

- **Contracts:** `NotificaSenderInterface` (`canale` , `invia`)
- **Services:** `NotificaService` (`notificaOperatore` , `notifica` , `inviaSuCanale`)
- **Implementations:**
 - `TelegramSender` (via `irazasyed/telegram-bot-sdk`)
 - `EmailSender` (via Laravel Mail)
 - `BrowserPushSender` (via `minishlink/web-push`)
- **Rules:** `CanaleSceltaRule` (seleziona canali per priorità)
- **Templates:** `RitardoSpedizione` , `SottoSogliaCarta`
- **Enums:** `CanaleNotifica` (Telegram/Email/BrowserPush), `PrioritaNotifica` (Bassa/Media/Alta/Critica)

Onda

Path: `app/Modules/Onda/` **Scopo:** Integrazione ERP Onda (SQL Server) → MES (MySQL).
Pattern architettura DDD modulare: isola coupling, prepara sostituzione futura.

- **Contracts:** `OndaErpInterface`
- **Adapters:** `OndaErpAdapter` (I/O SQL Server, 5 query estratte da `OndaSyncService` legacy)
- **Services:**
 - `OrdineSyncService::sync()` — Sync ordini produzione (`TipoDocumento=2`)
 - `ClienteSyncService` — Ricognizione clienti su `cliente_nome`
 - `CommessaSyncService::sync()` — Sync singola commessa (no filtro data)
- **ValueObjects:** `CodiceCommessaOnda::from()` (parser NNNNNNN-AA), `DocumentoOnda`
- **Enums:** `TipoDocumentoOnda` (OrdineProduzione=2, DdtVendita=3, DdtFornitore=7)
- **Events:** `OrdineSincronizzato(ordine_id, tipo, data)` , `ClienteSincronizzato(cliente_nome)`

Operatori

Path: `app/Modules/Operatori/` **Scopo:** Punto unico controlli autorizzazione MES. Matrice ruolo→permessi (statico) + regole contestuali (es. operatore modifica solo fasi reparto proprio).

- **Services:** `PermessiService` (`check` con context, `authorize` throws `AuthorizationException`)
- **Rules:** `AssegnazioneFaseRule` , `PermessiMatrix` (lookup statico ruolo→[Permesso])
- **Enums:**
 - `RuoloOperatore` (admin, owner, capo_reparto, operatore, pre stampa, spedizione, owner_readonly, fiery_contatori)
 - `Permesso` (ModificaFase, AvviaFase, TerminaFase, GestisciMagazzino, ...)

Presenze

Path: `app/Modules/Presenze/` **Scopo:** Gestione presenze operatori basata NetTime (server presenze, share file system). Calcolo ore lavorate, assenze, compatibilità reparto. Art. 4 Statuto Lavoratori.

- **Contracts:** `TimbratureSourceInterface`
- **Adapters:** `NetTimeShareAdapter` , `ManualeAdapter` (in-memory fallback testing)
- **Services:**
 - `PresenzeService` — Stato giorno + lookup oggi
 - `TimbratureSyncService` — Sync file BKP ogni 5 min
 - `CalcoloOreService` — Ore giorno/settimana/mese + pause
 - `AssenzeService` — CRUD assenze (`presenze_assenze`)
- **Rules:**
 - `ValidazioneOrarioRule` (6-22 lun-ven, XL106 24h)
 - `CalcoloStraordinariRule` (>40h=25%, >48h=35%)
 - `CompatibilitaTurnoReparto`
- **ValueObjects:** `PeriodoPresenza` , `BadgeOperatore` (matricola 6 cifre), `OreLavorate`
- **Enums:** `TipoTimbratura` (IN/OUT/PAUSA/RIENTRO), `TipoAssenza` (FERIE/MALATTIA/PERMESSO/SCIOPERO...), `StatoPresenza` (PRESENTE/IN_PAUSA/ASSENTE)
- **Events:** `TimbraturaRegistrata` , `OperatoreNonTimbrato` (alert >30min), `StraordinariSuperati`

Prinect

Path: `app/Modules/Princt/` **Scopo:** Integrazione Heidelberg Prinect Pressroom Manager (XL106). Pattern architettura DDD modulare verso `PrinctService` legacy + adapter `Stampa\PrinctAdapter`.

- **Contracts:** `PrinctApiInterface`
- **Adapters:** `PrinctHttpAdapter` (HTTP + Basic Auth)
- **Services:**
 - `PrinctJobsService` (`getWorkstepsStampa` , `stampaConfermata` , `jobIdToCommessa` , `commessaToJobId`)
 - `PrinctInkService::getConsumo()` — `inkConsumption` + cache 1h
 - `PrinctAccountingService::getTempi()` — Tempi avviamento/esecuzione da activities
 - `PrinctAutoTerminaService` (`deveTerminare` >4h inattiva, `deveRipristinare` se attività recenti)
- **ValueObjects:** `WorkstepId` (jobId, workstepId), `TempiStampa` (avviamento, esecuzione sec), `ConsumoInchiostro` (CMYK + spot grammi)
- **Enums:** `StatoWorkstep` (WAITING/RUNNING/COMPLETED/ABORTED)
- **Events:** `WorkstepAvviato` , `WorkstepCompleted(jobId, workstepId, tempiStampa)`

Reparti

Path: `app/Modules/Reparti/` **Scopo:** Tipizzazione 12 reparti (tabella `reparti`) + calcolo capacità/turni per scheduler Mossa 37 + dashboard owner. Logica pura (no DB write).

- **Services:**
 - `RepartoService` (`tutti` , `bySlug` , `byId` , `byCodice` , cache 1h)
 - `TurniService::turniPerReparto()` — Orari per CodiceReparto + giorno
 - `CapacitaService::oreDisponibiliGiorno()` — Emette `RepartoCapacitaSuperata`
- **Rules:** `AssegnazioneFaseReparto::reparto()` (pure: fase → CodiceReparto), `CompatibilitaMacchinaReparto::reparto()`
- **ValueObjects:** `CapacitaReparto` (reparto, macchineAttive, oreMacchina), `OrarioTurno` (oraInizio, oraFine, turno) + `::mattina()` / `::pomeriggio()` / `::notte()`
- **Enums:**
 - `CodiceReparto` (PRESTAMPA, STAMPA_OFFSET, FINITURA, LOGISTICA, SPEDIZIONE, ESTERNO, ...) con `::id()` , `::nome()` , `::tipo()` ,

`::oreSettimana()`

- `TipoReparto` (PRESTAMPA, STAMPA, FINITURA, LOGISTICA, SPEDIZIONE, ESTERNO)

- **Events:** `RepartoCapacitaSuperata`

Reportistica

Path: `app/Modules/Reportistica/` **Scopo:** Aggregazioni cross-cutting (KPI, report ore, panoramiche reparti, fustelle, esterne, produttività operatori). Sostituisce ~600 righe di SQL inline da `DashboardAdminController` (1579 LOC) + `DashboardOwnerController` (1911 LOC). Read-only + cache deterministico.

- **Services:**

- `KpiService` (`tutti` , `unico`) — 6 KPI cached 5 min
- `ReportOreService` (`perCommessa` , `perTemplate`)
- `PanoramicaRepartiService` (`overview` , `caricoEOre`) — cached 10 min
- `EsterneReportService::commesseEsterne()` — cached 10 min
- `FustelleReportService` (`overview` , `repartiFustellaIds`) — cached 10 min
- `ProgressoCommesseService` (`avanzamento` , `isCompletata` , `avanzamentoBatch`)
- `ProduttivitaOperatoriService` (`topPerFasi` , `performanceCompleta`)

- **Rules:** `EfficienzaRule` (`calcola` , `badge`), `SforatureRule::sforate()` (filtra >20% tolleranza), `PrioritaReportRule::score()`

- **ValueObjects:** `PeriodoReport` , `KpiCard` , `DatasetGrafico::toChartJs()` , `RigaReport`

- **Enums:** `TipoKpi` (6 standard), `PeriodoAggregazione` (giorno/sett/mese/trim/sem/anno), `TipoGrafico` (line/bar/pie/doughnut)

- **Cache:** `ReportCache::KEY_KPI` (TTL 5 min), `KEY_REPORT_ORE` (15 min), `KEY_PANORAMICA` (10 min)

Scheduling

Path: `app/Modules/Scheduling/` **Scopo:** Core progetto **Mossa 37**: ordinamento prioritario fasi dashboard operatore, con propagazione cascata. 4 livelli lessicografici (disponibilità, urgenza reale, batch affinity, sequenza ciclo).

- **Services:**

- `PrioritaService` (`calcolaPriorita` → float ordinabile, `ordina` → Collection sorted DESC)
- `PropagazioneService::propagaDopoTermine()` — Cascata attivazione su `FaseTerminata`

- **Contracts:** `SchedulerEngineInterface::schedule()`

- **Rules:**

- `SequenzaFasiRule::puoIniziare()` — Fasi precedenti completate?
- `UrgenzaRule::urgenza()` — Giorni residui (negativo = ritardo)
- `BatchAffinityRule::calcolaBatchGroup()` — Setup compatibili ±5gg
- `SetupChangeRule` — Cambio config BOBST/Piegaincolla = 1h

- **Enums:** `LivelloPriorita` (Disponibilita, Urgenza, Batch, Sequenza) con `::peso()`

Spedizione

Path: `app/Modules/Spedizione/` **Scopo:** Tracking BRT + gestione DDT MES + note consegne bidirezionali (owner↔spedizione). Non modifica `BrtService` legacy, lo usa. Cache `brt_stato`, `brt_data_consegna` su `ddt_spedizioni`. Esclude multi-spedizioni esito BRT `-22`. Note con polling 15s.

- **Services:**

- `TrackingService` (`aggiornaStato` cache DDT, `aggiornaTutti` batch DDT aperti)
- `NoteConsegneService` (`aggiungi`, `nonLette`, `segnaLetta`)
- `DdtSyncService` (sync vendita), `DdtFornitoreSyncService`, `DdtLavorazioniSyncService`

- **Rules:** `RitardoRule::isInRitardo()` (scatta evento), `MultiSpedizioneRule` (filtra `-22`)

- **ValueObjects:** `CodiceTracking` (BRT), `IndirizzoDestinazione` (via/cap/comune/provincia)

- **Enums:** `StatoSpedizione` (Preparazione/Spedito/InTransito/Consegnato/Fallito), `EsitoBrt` (1, -22, ...)

- **Events:** `SpedizioneInRitardo(ddt_id, giorni_ritardo)` → `NotificaSpedizioneInRitardoListener`

Stampa

Path: `app/Modules/Stampa/` **Scopo:** Astrazione comune sopra Prinect (Heidelberg XL106 offset) + Fiery (HP Indigo digitale). Espone `StampaIntegrationInterface` unico. Service legacy (`PrinectService` , `FieryService`) restano in Anti-Corruption Layer Adapters.

- **Contracts:** `StampaIntegrationInterface` (`getId` , `getJobInStampa` , `getCopieFatte` , `getTempiAvviamentoEsecuzione`)
 - **Adapters:** `PrinectAdapter` implements `StampaIntegrationInterface` , `FieryAdapter` implements `StampaIntegrationInterface`
 - **Services:** `StampaQuotaService::quotaResidua()` (tiratura – fatte)
 - **Rules:** `AvviamentoRule` (tempi setup), `CalcoloOreStampaRule`
 - **ValueObjects:** `Tiratura` (numero copie)
 - **Enums:** `TipoStampa` (Offset/Digitale)
-

Pattern comuni

Eventi cross-modulo

Tutti i moduli comunicano via Laravel Events:

Evento	Modulo dispatcher	Listener (modulo)
FaseAvviata	Fasi	Tracciamento (Audit)
FaseTerminata	Fasi	PropagazioneFasiSuccessive (Scheduling), NotificaCommessaCompletata (Commessa)
CommessaCompletata	Commessa	Notifica spedizione + audit
OrdineSincronizzato	Onda	Excel sync trigger
WorkstepCompleted	Prinect	Update fase MES
SottoSogliaEvento	Magazzino	NotificaSottoSoglia (Notifiche)
SpedizioneInRitardo	Spedizione	NotificaSpedizioneInRitardo (Notifiche)
FustellaPrelevata / Restituita	Fustelle	Audit
TimbraturaRegistrata	Presenze	Calcolo straordinari + audit
OperatoreNonTimbrato	Presenze	NotificaAlert (Notifiche)
StraordinariSuperati	Presenze	NotificaAlert + log
RepartoCapacitaSuperata	Reparti	Notifica owner

Adapter intercambiabili

Pattern adottato per tutte le integrazioni esterne:

```
Modulo/Contracts/XxxInterface
    ↑ implements
Modulo/Adapters/XxxHttpAdapter (prod)
Modulo/Adapters/XxxMockAdapter (test)
```

In `bootstrap/providers.php` / `AppServiceProvider`:

```
$this->app->bind(OndaErpInterface::class, OndaErpAdapter::class);
```

In testing: swap con `MockOndaAdapter` o Mockery.

ValueObjects con factory

VO immutabili con costruttore privato + factory `::from()`:

```
final readonly class CodiceCommessa {  
    private function __construct(public string $value) {}  
    public static function from(string $raw): self { /* validation */ }  
}
```

Rules pure

Rules sono **funzioni pure** testabili in isolamento (no DB, no Http):

```
final class UrgenzaRule {  
    public static function urgenza(Carbon $dataConsegna, Carbon $oggi): float  
    {  
        return $oggi->diffInDays($dataConsegna, false); // negativo = ritardo  
    }  
}
```

Permessi centralizzati

Tutti i moduli usano `PermessiService` (modulo Operatori) per autorizzazione:

```
app(PermessiService::class)->authorize(  
    $operatore,  
    Permessi::ModificaFase,  
    ['fase' => $fase]  
);
```

Cache standardizzata

Moduli read-heavy (Reportistica, Fustelle, Reparti) usano cache key strutturate:

```
ReportCache::KEY_KPI . ':' . hash('xxh64', json_encode($params))
```

TTL configurabili per tipo (KPI 5min, panoramica 10min, report ore 15min).

architettura DDD modulare backward compat

Wrapper compatibilità mantenuti per:

- Comandi artisan cron esistenti
- Controller legacy
- Script standalone (`import_commissa.php` , ecc.)
- Reflection-based callers

Esempio: `OndaSyncService::sincronizza()` chiama internamente `app(OrdineSyncService::class)->sync()` .

05. Integrazioni Esterne

Il Mossa 37 è integrato con **9 sistemi esterni**: 1 ERP, 2 macchine stampa, 1 corriere, 1 sistema presenze, 1 monitoring FV, 1 bot Telegram, 1 file Excel bidirezionale, 1 web push.

Diagramma di alto livello

```
flowchart LR
    MES["Mossa 37<br/>(Laravel 12)"]
    Onda["Onda ERP<br/>SQL Server"]
    Prinect["Prinect XL106<br/>REST API"]
    Fiery["Fiery V900<br/>REST API"]
    BRT["BRT corriere<br/>SOAP"]
    NetTime["NetTime<br/>CSV share"]
    SolarLog["Solar-Log 1000<br/>HTTP scrape"]
    Telegram["Telegram Bot API"]
    Excel["Excel<br/>sync_data.xlsx"]
    PushAPI["Web Push API<br/>VAPID"]

    Onda -->|SQL read| MES
    Prinect -->|REST poll| MES
    Fiery -->|REST poll| MES
    BRT <-->|SOAP query| MES
    NetTime -->|CSV pull| MES
    SolarLog -->|HTTP scrape| MES
    Telegram <-->|Webhook+Polling| MES
    Excel <-->|MD5 hash diff| MES
    MES -->|VAPID push| PushAPI
```

1. Onda ERP (SQL Server)

Voce	Valore
Tipo	ERP commerciale
Protocollo	SQL Server T-SQL via Laravel DatabaseManager
Connection	<code>onda</code> (config/database.php)
Credenziali	env <code>DB_ONDA_HOST</code> , <code>DB_ONDA_USERNAME</code> , <code>DB_ONDA_PASSWORD</code> , <code>DB_ONDA_DATABASE</code>
Direzione	MES legge (read-only)
Frequenza	Ogni ora (cron <code>onda:sync</code>)

File coinvolti:

- `app/Modules/Onda/Adapters/OndaErpAdapter.php` — Query SQL Server
- `app/Modules/Onda/Contracts/OndaErpInterface.php` — Contract
- `app/Modules/Onda/Services/OrdineSyncService.php` — Orchestrazione ordini
- `app/Modules/Onda/Services/CommessaSyncService.php` — Sync singola commessa
- `app/Console/Commands/SyncOnda.php` — Comando artisan
- `app/Services/OndaSyncService.php` — Wrapper compatibilità

Tabelle Onda lette:

Tabella	Uso
<code>ATTDocTeste</code>	Header documenti: <code>TipoDocumento</code> =2 ordini, 3 DDT vendita, 7 DDT fornitore
<code>ATTDocRighe</code>	Righe documenti: <code>CodArt</code> , <code>Descrizione</code> (live), <code>Qta</code> , <code>TipoRiga</code> (1=articolo, 2=fase)
<code>PRDDocTeste</code>	Header commesse produzione
<code>PRDDocFasi</code>	Fasi produzione: <code>CodFase</code> , <code>CodMacchina</code> , <code>QtaDaLavorare</code>
<code>PRDDocRighe</code>	Materiali commessa, costi
<code>OC_ATTDocRigheExt</code>	Specifiche supporto: <code>SuppBaseCM</code> , <code>SuppAltezzaCM</code> , <code>Resa</code> , <code>TotSupporti</code>
<code>PRDMacchinari</code>	Scarti previsti per macchina (<code>OC_FogliScartoIniz</code>)
<code>STDAnagrafiche</code>	Anagrafica clienti (<code>RagioneSociale</code>)

Query principali:

- `getOrdiniDal(Carbon $dal)` — Ordini con `DataRegistrazione >= $dal` (default 7gg)
- `getOrdiniPerCommessa(string $codCommessa)` — Singola commessa (no filtro data)

- `getScartiPrevistiPerMacchina()` — Mapping macchina → scarti

Comando artisan:

```
php artisan onda:sync          # sync ordini ultimi 7gg
php artisan onda:sync 0067339-26 # sync singola commessa
```

Note operative:

- MES **non scrive** mai su Onda (read-only)
- Descrizione preferita: `ATTDocRighe.Descrizione` (commerciale live) con fallback `OC_Descrizione` (PRD stale)
- Lookup ordine: match commessa + cod_art + descrizione, fallback su commessa+cod_art per gestire revisioni OC
- Eventi: `OrdineSincronizzato`, `ClienteSincronizzato`

2. Prinect (Heidelberg XL106, REST)

Voce	Valore
Tipo	Workflow stampa offset
Protocollo	REST + Basic Auth
Base URL	env <code>PRINECT_API_URL</code>
Direzione	MES legge (read-only)
Frequenza polling	Ogni 5 minuti (cron <code>princt:sync-attivita</code>), on-demand dashboard 15-30s
Cache	60s jobs/accounting, 30s server status, fallback stale 30min

File coinvolti:

- `app/Http/Services/PrinctService.php` — Wrapper legacy
- `app/Modules/Princt/Adapters/PrinctHttpAdapter.php` — Adapter modulare
- `app/Modules/Princt/Contracts/PrinctApiInterface.php`
- `app/Modules/Princt/Services/PrinctJobsService.php`
- `app/Modules/Princt/Services/PrinctInkService.php`
- `app/Modules/Princt/Services/PrinctAccountingService.php`
- `app/Modules/Princt/Services/PrinctAutoTerminaService.php`

Endpoint principali:

Endpoint	Uso
GET /rest/device	Lista device + status
GET /rest/devicegroup	Device grouping
GET /rest/job/{jobId}	Dettaglio job
GET /rest/job/{jobId}/element	Elementi job
GET /rest/job/{jobId}/workstep/{wsId}/activity	Attività workstep (timestamps)
GET /rest/job/{jobId}/workstep/{wsId}/preview	Anteprima foglio stampa
GET /rest/job/{jobId}/workstep/{wsId}/quality	Quality data (scarti)
GET /rest/job/{jobId}/workstep/{wsId}/ink	Consumo inchiostro CMYK
GET /rest/employee	Operatori Prinect
GET /rest/version	API version

Dati letti:

- Device activity (CPU, memory, job count)
- Job metadata (id, status, modified date)
- Workstep execution (activities, previews, quality scores, ink usage)
- Mapping `jobId` → `commessa` : ltrim primi 7 char commessa, numerico

Comando artisan:

```
php artisan prinect:sync-attivita # storico 7gg
```

Note operative:

- Auto-termina: workstep COMPLETED senza `actualStartDate` → terminate fase MES (fix bug commessa 66811)
- Auto-ripristino: fase MES stato=3 con attività Prinect recenti → ripristina stato=2
- Device ID XL106: env `PRINECT_DEVICE_XL106_ID` (default 4001)

3. Fiery (Canon V900, REST)

Voce	Valore
Tipo	Workflow stampa digitale
Protocollo	REST + Cookie session auth
Base URL	<code>config('fiery.host')</code>
API version	v5
Direzione	MES legge (read-only)
Frequenza	Ogni minuto (cron <code>fiery:sync</code>)
Cache	30s status, 60s jobs/accounting, 1h info, 24h version

File: `app/Http/Services/FieryService.php`

Endpoint:

Endpoint	Uso
<code>POST /live/api/v5/login</code>	Auth (session cookie)
<code>GET /live/api/v5/server/status</code>	Health server (ready/processing/error)
<code>GET /live/api/v5/jobs</code>	Job queue (id, state, copies, sheets)
<code>GET /live/api/v5/accounting</code>	Ledger per commessa
<code>GET /live/api/v5/consumables</code>	Toner CMYK + carta + waste
<code>GET /live/api/v5/info</code>	Server info (firmware, capabilities)

Dati letti:

- Server status, queue job
- Accounting per commessa (fogli, copie, history)
- Consumables (toner, carta, waste toner)

Comandi artisan:

Comando	Frequenza
<code>php artisan fiery:sync</code>	Ogni minuto
<code>php artisan fiery:snapshot-contatori</code>	Feriali 16:55 (pre-shutdown 17:00)
<code>php artisan fiery:export-contatori --mese-corrente --email={REPORT_CONTATORI_T0}</code>	Ultimo del mese 17:00

Note operative:

- Fallback stale cache 30 minuti se chiamate live falliscono (resilienza dashboard)
- Categorie contatori (`fiery_contatori`): grouping fogli per categoria/mese/anno

4. BRT (SOAP corriere)

Voce	Valore
Tipo	Corriere espresso
Protocollo	SOAP 1.1 su HTTPS (TLS)
Auth	<code>CLIENTE_ID</code> userID in SOAP body
Direzione	MES query tracking
Frequenza	On-demand (click dashboard)

File: `app/Http/Services/Brtservice.php`

WSDL:

- `https://<endpoint>
BRT>/web/GetIdSpedizioneByRMAService/GetIdSpedizioneByRMA?wsdl`
- `https://<endpoint>
BRT>/web/BRT_TrackingByBRTshipmentIDService/BRT_TrackingByBRTshipmentID?wsdl`

Dati letti:

- Shipment ID da DDT (`RIFERIMENTO_MITTENTE_ALFABETICO` o `RIFERIMENTO_MITTENTE_NUMERICO`)
- Tracking completo: data/ora consegna, destinatario, firma, peso/colli, eventi
- Bolla: indirizzi mittente/destinatario, servizio, porto, filiale

Note operative:

- WSDL rewrite HTTP → HTTPS (bug BRT legacy WSDL)
- Temp WSDL file creato con permessi 0600, auto-clean shutdown
- Search strategy multi-anno: alfabetico anno corrente → anno prec → numerico (gestisce DDT cross-anno)
- Esito `-22` = MULTI-spedizione (filtrata da email ritardi e da alert)
- SSL verify: env `BRT_VERIFY_SSL` (default false in dev, true in prod)

5. NetTime (presenze)

Voce	Valore
Tipo	Sistema biometrico presenze
Server NetTime	<IP gestione presenze> (Antonio IT)
Share export	\\<IP ERP>\nettime_timbrature\
Protocollo	Lettura CSV da Windows share
Direzione	MES legge (one-way pull)
Frequenza	Ogni minuto feriali 05-23

File:

- `app/Modules/Presenze/Adapters/NetTimeShareAdapter.php`
- `app/Modules/Presenze/Services/TimbratureSyncService.php`
- `app/Console/Commands/Presenze*.php`

Comandi artisan:

Comando	Frequenza
<code>php artisan presenze:sync</code>	Ogni minuto feriali 05-23
<code>php artisan presenze:export-excel</code>	Ogni 15 min feriali 05-23

Env config:

- `NETTIME_SHARE_USER` , `NETTIME_SHARE_PASS` (credenziali share)
- TODO: `NETTIME_PC_USER` , `NETTIME_PC_PASS` , `NETTIME_EXPORT_CMD` (per cron remoto su PC NetTime)

Note operative:

- Procedura sindacale art. 4 Statuto Lavoratori richiesta prima di rollout produttività individuale
- Informativa GDPR art. 13 firmata da ogni dipendente

6. Solar-Log 1000 (monitoring FV)

Voce	Valore
Tipo	Monitoring impianto fotovoltaico
Dispositivo	Solar-Log 1000, firmware 2.8.3 (2013)
Impianto	180 kWp, 7 inverter Power-One PVI-TRIO
IP locale	<IP gestione presenze> (MAC 00-19-99-df-96-b0)
Portale cloud	https://solarlog-portal.it
Direzione	MES legge (scrape)
Stato	Integrazione in corso

Endpoint portale:

- Login: `POST https://solarlog-portal.it/974821.html`
(username/password/action=login)
- Pagina rendimenti: `https://solarlog-portal.it/emulated_yieldov_3650.html` (con session cookie)

Note operative:

- API locale `/getjp` non supportata (firmware 2013 troppo vecchio)
- IIS sul PC dedicato intercetta porta 80 (conflitto, non risolvibile)
- FTP `websu.solarlog-portal.it` ritorna errore 530 (IP-restricted)
- Soluzione: scrape pagina rendimenti via login + session cookie

7. Telegram Bot

Voce	Valore
Tipo	Bot messaggistica (input bolle magazzino + alert)
Protocollo	HTTPS Webhook + Bot API long polling
Token	env <code>TELEGRAM_BOT_TOKEN</code>
Direzione	Bidirezionale
Frequenza	Real-time webhook / 30s polling

File:

- `app/Http/Controllers/TelegramWebhookController.php`

- `app/Console/Commands/TelegramPoll.php`
- `app/Services/BollaAIService.php` — Claude Vision API per OCR foto bolle

Endpoint MES:

- `POST /telegram/webhook/{secret}` — Webhook con HMAC-SHA256 constant-time verify

Comando artisan:

```
php artisan telegram:poll --timeout=30 # long polling 30s
php artisan telegram:poll --once      # single cycle
```

Comandi bot: `/start`, `/help`, `/id`, `/ping`, `/status`, `/giacenza`, `/articoli`, `/alert`, `/movimenti`, `/carico`

AI integration:

- `BollaAIService::analizzaBolla()` via Claude Vision API
- Input: foto bolla via Telegram
- Output: parsing campi (articolo, qta, lotto, fornitore) → registrazione movimento magazzino auto
- Env: `ANTHROPIC_API_KEY`

Deploy:

- Windows service via `nssm` chiamato `Mossa37TelegramBot` (auto-restart)
- Webhook e polling **mutuamente esclusivi**: rimuovere webhook prima di avviare polling

Sicurezza:

- Secret webhook string ≥ 16 caratteri (regex validate)
- Verifica constant-time `hash_equals()` (HMAC-SHA256 su JSON body)

8. Excel bidirezionale

Voce	Valore
Tipo	File scambio dati MES ↔ Excel utente
Path	env <code>EXCEL_SYNC_PATH</code> o <code>storage/app/excel_sync/sync_data.xlsx</code>
Protocollo	XLSX (PhpSpreadsheet) + MD5 hash detection
Direzione	Bidirezionale
Frequenza	Cron ogni 2 min (<code>excel:sync</code>) + on-page-load

File:

- `app/Http/Services/ExcelSyncService.php` (legacy)
- `app/Modules/Documenti/Services/ExcelSyncService.php` (nuovo modulo)

State files (stessa cartella):

- `.last_sync_timestamp`
- `.last_sync_hash` (MD5)
- `.last_exported_ids`

Logica:

- **syncIn:** legge `sync_data.xlsx`, calcola MD5, confronta con `.last_sync_hash`
 - Se diverso: parsing colonne A-AJ, raggruppa per commessa, aggiorna campi `OrdineFase` corrispondenti
 - Riga eliminata da Excel = soft-delete fase MES
 - Propaga modifiche data consegna a tutti ordini stessa commessa
- **syncOut:** export current state DB → xlsx, aggiorna state files

Comando artisan:

```
php artisan excel:sync # ogni 2 minuti
```

9. Web Push (browser notifications)

Voce	Valore
Tipo	Browser push notifications
Protocollo	Web Push API + VAPID
Direzione	MES invia
Frequenza	On-event

File:

- `app/Http/Controllers/PushController.php`
- `app/Modules/Notifiche/Senders/BrowserPushSender.php`

Endpoint:

- `POST /push/subscribe` — Registra subscription
- `POST /push/unsubscribe` — Cancella
- `GET /push/vapid-key` — Public key VAPID

Tabella: `push_subscriptions`

Eventi inviati:

- Spedizione: nuova fase pronta consegna
- Owner: nota consegna nuova da spedizione
- Magazzino: giacenza sotto soglia
- Presenze: alert non-timbrato

Tabella riepilogativa

Sistema	Protocollo	Direzione	Frequenza	Comando artisan
Onda ERP	SQL Server	Read	Ogni ora	<code>onda:sync [commessa]</code>
Prinect	REST Basic	Read	5 min cron + on-demand	<code>prinect:sync-attivit�</code>
Fiery	REST Cookie	Read	1 min	<code>fiery:sync</code> , <code>fiery:snapshot-contatori</code> , <code>fiery:export-contatori</code>
BRT	SOAP/HTTPS	Query	On-demand	—
NetTime	CSV share	Pull	1 min feriali	<code>presenze:sync</code> , <code>presenze:export-excel</code>
Solar-Log	HTTP scrape	Read	TBD	TBD (in corso)
Telegram	Webhook + polling	Bidirezionale	Real-time / 30s	<code>telegram:poll</code>
Excel	XLSX + MD5	Bidirezionale	2 min	<code>excel:sync</code>
Web Push	VAPID	Send	On-event	—

Considerazioni multi-tenant per deploy partner

Aspetto	Da configurare per ogni tenant
Onda	Hostname + credenziali SQL Server (potrebbe NON essere Onda — sostituibile via swap Adapter)
Prinect	Base URL + Basic Auth + device ID XL106 (se cliente non ha XL106, modulo Prinect disattivato)
Fiery	Host + credenziali (se cliente non ha Fiery, modulo Fiery disattivato)
BRT	Cliente ID + credenziali (se cliente usa altro corriere → adapter custom)
NetTime	Path share + credenziali (sostituibile con altro timbratore via swap Adapter)
Telegram	Bot token dedicato per ogni tenant
Push	VAPID keys per ogni tenant

Tutte le integrazioni passano da **Contracts + Adapters** → per integrare un cliente con stack diverso (es. SAP invece di Onda) basta scrivere un nuovo Adapter senza toccare business logic.

06. Database

Overview

Voce	Valore
DBMS operativo	MySQL 8
Schema	mossa37
Migrations totali	115
Model Eloquent	28
Host	<host DB locale> ()
Charset	utf8mb4
Engine	InnoDB

DBMS aggiuntivo: **SQL Server Onda** (connection **onda**) — solo lettura, vedi **05-integrations.md**.

Diagramma ER (alto livello)

```
erDiagram
    ORDINI ||--o{ ORDINE_FASI : "ha"
    ORDINI }o--|| REPARTI : "appartiene"
    ORDINE_FASI }o--|| FASI_CATALOGO : "tipologia"
    ORDINE_FASI }o--|| OPERATORI : "assegnato"
    FASI_CATALOGO }o--|| REPARTI : "in"
    ORDINI ||--o{ DDT_SPEDIZIONI : "spedito"
    OPERATORI }o--o{ ORDINE_FASI : "pivot fase_operatore"
    OPERATORI }o--|| REPARTI : "principale"
    MAGAZZINO_ARTICOLI ||--o{ MAGAZZINO_GIACENZE : "in"
    MAGAZZINO_ARTICOLI ||--o{ MAGAZZINO_MOVIMENTI : "storico"
    MAGAZZINO_GIACENZE }o--|| MAGAZZINO_UBICAZIONI : "in"
    ORDINI }o--|| CLICHE_ANAGRAFICA : "usa"
    ORDINE_FASI }o--o| FUSTELLE : "usa"
    AUDIT_LOGS }o--o| USERS : "azione di"
```

I 28 Model

Model	Tabella	Soft Delete
Ordine	ordini	No
OrdineFase	ordine_fasi	Sì
FasiCatalogo	fasi_catalogo	No
Fase	fasi	No (legacy)
Operatore	operatori	No
Reparto	reparti	No
Assegnazione	assegnazioni	No
PausaOperatore	pausa_operatores	No
DdtSpedizione	ddt_spedizioni	No
NotaSpedizione	note_spedizione	No
MagazzinoArticolo	magazzino_articoli	No
MagazzinoGiacenza	magazzino_giacenze	No
MagazzinoMovimento	magazzino_movimenti	No
MagazzinoUbicazione	magazzino_ubicazioni	No
MagazzinoEtichetta	magazzino_etichette	No
Fustella	fustelle	No
ClicheAnagrafica	cliche_anagrafica	No (legacy)
Articolo	articoli	No
User	users	No
CostoReparto	costi_reparti	No
PrinectAttivita	prinect_attivita	No
ChatMessage	chat_messages	No
ContatoreStampante	contatori_stampanti	No
EanProdotto	ean_prodotti	No
FieryAccounting	fiery_accounting	No
OperatoreToken	operatore_tokens	No

Model	Tabella	Soft Delete
NotaTurno	note_turni	No
PushSubscription	push_subscriptions	No

AuditLog (modulo Audit) usa tabella `audit_logs` ma non è Model Eloquent — accesso via service.

Tabelle critiche (dettaglio)

`ordini` — Ordini cliente


```

id                BIGINT PK AUTO_INCREMENT
commessa          VARCHAR(255) INDEX
cliente_nome      VARCHAR(255) INDEX
cod_art           VARCHAR(255) NULL
descrizione       TEXT
qta_richiesta     INTEGER
qta_prodotta      INTEGER DEFAULT 0
um               VARCHAR(10) DEFAULT 'FG'
stato             TINYINT DEFAULT 0          -- 0=non iniziato, 1=in lav, 2=terminato
priorita          INTEGER DEFAULT 0
data_registrazione DATE
data_prevista_consegna DATE NULL
pronto_consegna   BOOLEAN DEFAULT false
note             TEXT
ore_lavorate      DECIMAL(6,2) NULL
timeout_macchina  DECIMAL(6,2) NULL
cliche_numero     INTEGER NULL FK->cliche_anagrafica(numero)
cliche_match_type VARCHAR NULL
cliche_matched_at DATETIME NULL
ddt_vendita_id    BIGINT NULL
numero_ddt_vendita VARCHAR NULL
vettore_ddt       VARCHAR NULL
qta_ddt_vendita   DECIMAL NULL
cod_carta         VARCHAR NULL
carta            VARCHAR NULL
qta_carta         INTEGER NULL
UM_carta          VARCHAR(10) NULL
supp_base_cm      INTEGER NULL
supp_altezza_cm   INTEGER NULL
resa            INTEGER NULL
tot_supporti      INTEGER NULL
valore_ordine     DECIMAL NULL
costo_materiali   DECIMAL NULL
note_prestampa    TEXT
responsabile      TEXT
commento_produzione TEXT
note_fasi_successive TEXT (JSON array)
ordine_cliente    VARCHAR NULL
created_at, updated_at TIMESTAMP

```

Relazioni Eloquent:

```
hasMany(OrdineFase)
hasMany(Articolo)
hasMany(Assegnazione)
hasMany(PausaOperatore)
belongsTo(Reparto)
belongsTo(ClicheAnagrafica, 'cliche_numero', 'numero')
hasMany(DdtSpedizione)
hasManyThrough(Operatore, OrdineFase)
```

Indici: `commessa`, `cliente_nome`

`ordine_fasi` — Fasi di produzione (CORE)

Tabella con **SOFT DELETE** (`deleted_at`).

```

id                BIGINT PK
ordine_id         BIGINT FK→ordini ON DELETE CASCADE
fase_catalogo_id BIGINT FK→fasi_catalogo
operatore_id      BIGINT FK→operatori NULL
fase              VARCHAR(255)
stato             TINYINT DEFAULT 0          -- 0=caricato, 1=pronto, 2=avviato,
                                                -- 3=terminato, 4=consegnato, 5=esterno

data_inizio       TIMESTAMP NULL
data_fine         TIMESTAMP NULL
reparto           VARCHAR NULL
qta_prod          INTEGER DEFAULT 0
scarti            INTEGER NULL
tiro              DECIMAL NULL
qta_fase          INTEGER NULL
um               VARCHAR(10) NULL
note              TEXT
priorita          INTEGER DEFAULT 0
priorita_manuale  BOOLEAN NULL
esterno           BOOLEAN DEFAULT false
terminata_manualmente BOOLEAN DEFAULT false
manuale           BOOLEAN DEFAULT false
ore               DECIMAL NULL
tipo_consegna     VARCHAR NULL
ddt_fornitore_id BIGINT NULL
segnacollo_brt    VARCHAR NULL
scarti_previsti   INTEGER NULL
riaperta_at       TIMESTAMP NULL
qta_prod_at_riapertura INTEGER NULL
deleted_at        TIMESTAMP NULL          -- soft delete

-- Colonne Mossa 37 (scheduler)
sequenza          INTEGER
disponibile       BOOLEAN DEFAULT false
disponibile_m37   BOOLEAN DEFAULT false
urgenza_reale     DECIMAL(8,2) NULL
fascia_urgenza    INTEGER NULL
giorni_lavoro_residuo DECIMAL(8,2) NULL
batch_key         VARCHAR NULL
sequenza_m37      INTEGER

```

```
priorita_m37          INTEGER
sched_posizione       VARCHAR NULL
sched_macchina        VARCHAR NULL
sched_inizio          DATETIME NULL
sched_fine            DATETIME NULL
sched_setup_h         DECIMAL NULL
sched_setup_tipo      VARCHAR NULL
sched_batch_group     VARCHAR NULL
sched_calcolato_at    TIMESTAMP NULL

created_at, updated_at  TIMESTAMP
```

Relazioni:

```
belongsTo(Ordine)
belongsTo(Operatore)
belongsTo(FasiCatalogo)
belongsToMany(Operatore, 'fase_operatore', withPivot('data_inizio','data_fine','secondi_pausa')
hasOneThrough(Reparto, FasiCatalogo)
```

Casts: boolean (`disponibile` , `disponibile_m37` , `priorita_manuale` , `esterno`), integer (`sequenza` , `sequenza_m37` , `fascia_urgenza`)

Indici critici: `ordine_id` , `fase_catalogo_id` , `operatore_id` , `stato` , `deleted_at` , `priorita_m37` (perf indices da migrations 2026-04-20 + 2026-05-06)

`fasi_catalogo` — Catalogo fasi disponibili

```
id          BIGINT PK
nome        VARCHAR(255)
reparto_id  BIGINT FK→reparti
pronto_consegna  DECIMAL(8,2) NULL
avviamento  DECIMAL(8,2) NULL
copie_ora    INTEGER NULL
unita_misura  VARCHAR NULL
created_at, updated_at  TIMESTAMP
```

Relazione: `belongsTo(Reparto)`

`operatori` — Operatori (auth guard `operatore`)

```
id                BIGINT PK
nome              VARCHAR(255)
cognome           VARCHAR(255) NULL
codice_operatore  VARCHAR(255) UNIQUE    -- alfanumerico, es. "RB", "MIR"
reparto           VARCHAR NULL          -- legacy
reparto_id        BIGINT FK→reparti NULL
ruolo             VARCHAR DEFAULT 'operatore'
                  -- operatore|superadmin|owner|admin|
                  -- pre stampa|spedizione|fiery_contattori|
                  -- owner_readonly
attivo            BOOLEAN DEFAULT true
password          VARCHAR(255) NULL
remember_token    VARCHAR NULL
created_at, updated_at  TIMESTAMP
```

Relazioni:

```
hasMany(Assegnazione)
hasMany(PausaOperatore)
belongsToMany(OrdineFase, 'fase_operatore', withPivot('data_inizio','data_fine','secondi_pausa'))
belongsTo(Reparto)
belongsToMany(Reparto, 'operatore_reparto')
hasMany(MagazzinoMovimento)
```

Auth Guard: `operatore` (config/auth.php)

`reparti` — Reparti produttivi

```
id                BIGINT PK
nome              VARCHAR(255) UNIQUE
codice            VARCHAR NULL
created_at, updated_at  TIMESTAMP
```

Relazioni: `hasMany(FasiCatalogo)`, `hasMany(Fase)`, `hasMany(CostoReparto)` orderByDesc(`valido_dal`)

12 reparti standard: pre stampa, stampa offset, digitale, finitura digitale, plastica, plastica lux, caldo, fustella, piegaincolla, legatoria, spedizione, esterno.

magazzino_articoli — Anagrafica magazzino

```
id                BIGINT PK
codice            VARCHAR(255) UNIQUE
descrizione       VARCHAR(255)
categoria         VARCHAR NULL -- carta|foil|scatoloni|inchiostro|vernici
formato           VARCHAR NULL
grammatura        INTEGER NULL
spessore          DECIMAL(5,3) NULL
um               VARCHAR(10) DEFAULT 'fg'
soglia_minima     DECIMAL(2) DEFAULT 0
fornitore         VARCHAR NULL
certificazioni    VARCHAR NULL
attivo            BOOLEAN DEFAULT true
created_at, updated_at  TIMESTAMP
```

Metodi:

- `giacenzaTotale(): int` — somma giacenze tutte ubicazioni
- `sottoSoglia(): bool` — controllo soglia minima

Relazioni: `hasMany(MagazzinoGiacenza)` , `hasMany(MagazzinoMovimento)` ,
`hasMany(MagazzinoEtichetta)`

magazzino_giacenze — Giacenze per ubicazione

```
id                BIGINT PK
articolo_id        BIGINT FK→magazzino_articoli
ubicazione_id     BIGINT FK→magazzino_ubicazioni NULL
quantita          INTEGER DEFAULT 0
lotto             VARCHAR NULL
data_ultimo_carico DATE NULL
data_ultimo_scarico DATE NULL
created_at, updated_at  TIMESTAMP

UNIQUE(articolo_id, ubicazione_id, lotto)
```

magazzino_movimenti — Storico movimenti

```
id                BIGINT PK
articolo_id       BIGINT FK→magazzino_articoli
ubicazione_id    BIGINT FK→magazzino_ubicazioni NULL
tipo             VARCHAR -- scarico|carico|reso|rettifica
quantita         DECIMAL(2)
giacenza_dopo    DECIMAL(2)
lotto            VARCHAR NULL
fornitore        VARCHAR NULL
commessa         VARCHAR NULL
fase             VARCHAR NULL
operatore_id     BIGINT FK→operatori NULL
note             TEXT NULL
foto_bolla       VARCHAR NULL -- path foto Telegram
ocr_raw          TEXT NULL -- output Claude Vision OCR
created_at, updated_at TIMESTAMP
```

fustelle — Anagrafica fustelle (nuova)

```
id                BIGINT PK
codice            VARCHAR(20) UNIQUE -- F-NNNNN-X
tipo             ENUM('PIANA','ROTATIVA','TRINCIATURA','RILIEVO')
stato            ENUM('PREPARAZIONE','PRONTA','IN_USO','ARCHIVIATA')
dimensione_mm_x  UNSIGNED INT NULL
dimensione_mm_y  UNSIGNED INT NULL
spessore_mm      DECIMAL(5,2) NULL
posizione_magazzino VARCHAR(50) NULL
note             TEXT NULL
created_at, updated_at TIMESTAMP
```

Casts: `TipoFustella` enum, `StatoFustella` enum.

cliche_anagrafica — Legacy fustelle/cliché

```
id                BIGINT PK
numero            INTEGER UNIQUE
descrizione_raw   VARCHAR(500)
qta               INTEGER NULL
scatola           INTEGER NULL
note              VARCHAR(500) NULL
created_at, updated_at  TIMESTAMP
```

Tabella **legacy**. La nuova è **fustelle** (sopra). Coesistono in attesa di migrazione completa.

ddt_spedizioni — DDT MES

```
id                BIGINT PK
onda_id_doc       UNSIGNED BIGINT
numero_ddt        VARCHAR(20)
data_ddt          DATE NULL
vettore           VARCHAR(100) NULL
cliente_nome      VARCHAR(150) NULL
commessa          VARCHAR(20)
ordine_id         UNSIGNED BIGINT FK→ordini ON DELETE SET NULL
qta               DECIMAL(10,2) NULL
brt_stato         VARCHAR NULL
brt_data_consegna DATE NULL
brt_destinatario  VARCHAR NULL
brt_colli         VARCHAR NULL
brt_cache_at      DATETIME NULL
created_at, updated_at  TIMESTAMP

UNIQUE(onda_id_doc, commessa)
```

Relazione: **belongsTo(Ordine)**

audit_logs — Audit log eventi


```

id          BIGINT PK
user_id     UNSIGNED BIGINT NULL
user_name   VARCHAR(100) NULL
action      VARCHAR(50)  -- login|logout|update|create|delete|sync|export|read|failed
model       VARCHAR(100) NULL
model_id    UNSIGNED BIGINT NULL
old_values  JSON NULL
new_values  JSON NULL
ip          VARCHAR(45) NULL
user_agent  VARCHAR(500) NULL
extra       TEXT NULL
created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP

```

Indici: `user_id`, `action`, `model`, `created_at`, composite (`model`, `model_id`)

Retention automatica: comando `audit:pulisci` giornaliero 03:00, default 90 giorni
(configurabile per livello sicurezza: Normale 3yr, Sensibile 2yr, Critico 7yr).

`users` — Admin (auth guard `web`)

```

id          BIGINT PK
name        VARCHAR(255)
email       VARCHAR(255) UNIQUE
password    VARCHAR(255)
email_verified_at  TIMESTAMP NULL
remember_token  VARCHAR NULL

-- 2FA
2fa_secret  VARCHAR NULL
2fa_recovery_codes  TEXT NULL  -- JSON 8 codici 4-4 one-time
2fa_enabled_at  TIMESTAMP NULL

created_at, updated_at  TIMESTAMP

```

Pivot tables

`fase_operatore`

Pivot OrdineFase ↔ Operatori (relazione multi-a-molti).

```
id                BIGINT PK
ordine_fase_id    BIGINT FK→ordine_fasi
operatore_id      BIGINT FK→operatori
data_inizio       TIMESTAMP NULL
data_fine         TIMESTAMP NULL
secondi_pausa     INTEGER DEFAULT 0
created_at, updated_at  TIMESTAMP
```

assegnazioni

Pivot legacy Ordini ↔ Operatori.

```
id                BIGINT PK
ordine_id         BIGINT FK
operatore_id      BIGINT FK
created_at, updated_at  TIMESTAMP
```

operatore_reparto

Pivot Operatori ↔ Reparti (operatore multi-reparto).

```
id                BIGINT PK
operatore_id      BIGINT FK
reparto_id        BIGINT FK
created_at, updated_at  TIMESTAMP
```

Soft Delete

Solo tabella **ordine_fasi** usa soft delete (**deleted_at** nullable).

Motivazione: una fase può essere "eliminata" dall'owner per errore o cambiamento ordine cliente, ma serve audit storico per:

- Tracciabilità (compliance)
- Riapertura accidentale
- Ricostruzione cronologia

Helper:

```

OrdineFase::withTrashed()    // include soft-deleted
OrdineFase::onlyTrashed()   // solo soft-deleted
$fase->restore()             // ripristina
$fase->forceDelete()         // hard delete (riservato admin)

```

Mossa 37 (scheduler) — 17 colonne extra in `ordine_fasi`

Per supportare lo scheduler con propagazione fasi e ottimizzazione setup:

Colonna	Tipo	Uso
<code>sequenza</code>	INT	Sequenza standard ciclo (config <code>fasi_priorita.php</code>)
<code>disponibile</code>	BOOL	Fase può essere avviata (fasi precedenti terminate)
<code>disponibile_m37</code>	BOOL	Disponibilità calcolata da scheduler Mossa 37
<code>urgenza_reale</code>	DECIMAL(8,2)	Giorni residui (negativo = ritardo)
<code>fascia_urgenza</code>	INT	Banda urgenza (1=critico, 5=non urgente)
<code>giorni_lavoro_residuo</code>	DECIMAL	Giorni lavoro residui per completamento
<code>batch_key</code>	VARCHAR	Chiave raggruppamento batch (setup affinity ±5gg)
<code>sequenza_m37</code>	INT	Posizione ottimizzata da scheduler
<code>priorita_m37</code>	INT	Priorità finale (4 livelli lessicografici)
<code>sched_posizione</code>	VARCHAR	Posizione cronologica
<code>sched_macchina</code>	VARCHAR	Macchina assegnata
<code>sched_inizio</code>	DATETIME	Inizio pianificato
<code>sched_fine</code>	DATETIME	Fine pianificata
<code>sched_setup_h</code>	DECIMAL	Ore setup pianificate
<code>sched_setup_tipo</code>	VARCHAR	Tipo setup (cambio config BOBST/PI)
<code>sched_batch_group</code>	VARCHAR	Gruppo batch finale
<code>sched_calcolato_at</code>	TIMESTAMP	Quando scheduler ha calcolato

Migrations recenti (ultime 10)

Data	Migration
2026-05-06	add_perf_indices_ordine_fasi
2026-05-05	add_scarico_carta_to_ordine_fasi
2026-04-30	add_qta_prod_at_riapertura_to_ordine_fasi
2026-04-30	add_riaperta_at_to_ordine_fasi
2026-04-20	add_perf_indexes_to_ordine_fasi
2026-04-17	add_tiro_to_ordine_fasi
2026-04-17	add_cliche_to_ordini
2026-04-17	create_cliche_anagrafica
2026-04-16	magazzino_on_delete_restrict
2026-04-16	fix_magazzino_audit

Note architetturali

Dual-auth: 2 tabelle utenti

- `users` → guard `web` (admin, accesso `/admin/*`)
- `operatori` → guard `operatore` (staff produzione, accesso `/operatore/*`, `/produzione/*`, `/spedizione/*`)

Configurazione in `config/auth.php`:

```
'guards' => [  
    'web' => ['driver' => 'session', 'provider' => 'users'],  
    'operatore' => ['driver' => 'session', 'provider' => 'operatori'],  
],
```

Cliché dual

- `cliche_anagrafica` (legacy Excel fustellificio, FK `ordini.cliche_numero`)
- `fustelle` (nuova tabella tipizzata, modulo Fustelle)

Migrazione in corso da legacy a nuova.

DdtSpedizioni come bridge

Bridge tra Onda DDT e ordini MES. La unique `(onda_id_doc, commessa)` evita duplicati durante sync. La FK `ordine_id` è ON DELETE SET NULL: se l'ordine viene cancellato, il DDT mantiene tracking BRT cached.

Magazzino: ON DELETE RESTRICT

Le FK in `magazzino_giacenze`, `magazzino_movimenti` verso `magazzino_articoli` sono RESTRICT — non si può eliminare un articolo se ha giacenze/movimenti (audit + compliance).

`audit_logs` standalone

`user_id` è UNSIGNED BIGINT ma **NON è FK** — log conservato anche dopo eliminazione utente (compliance).

07. Flussi

Sequence diagrams + spiegazione step-by-step dei flussi business critici di Mossa 37.

Flusso 1: Sync Onda ERP → MES

Frequenza: **ogni ora** (cron `php artisan onda:sync`).

```
sequenceDiagram
    participant Cron
    participant Wrapper as OndaSyncService (legacy)
    participant Sync as OrdineSyncService
    participant Adapter as OndaErpAdapter
    participant OndaDB as SQL Server Onda
    participant MySQL
    participant Events

    Cron->>Wrapper: php artisan onda:sync
    Wrapper->>Sync: sync()
    Sync->>Adapter: getOrdiniDal(now-7gg)
    Adapter->>OndaDB: SELECT ATTDocTeste t<br/>JOIN PRDDocTeste p<br/>JOIN PRDDocFasi f<br/>OU
    OndaDB-->>Adapter: righe
    Adapter-->>Sync: array
    Sync->>Sync: raggruppa per commessa+cod_art+desc

    loop per ogni gruppo
        Sync->>MySQL: lookup Ordine (commessa+cod_art+desc)
        alt non trovato
            Sync->>MySQL: fallback lookup commessa+cod_art
        end
        alt esistente
            Sync->>MySQL: update campi (cliente, data, carta, ecc.)
            Sync->>MySQL: update descrizione se diversa
        else nuovo
            Sync->>MySQL: create Ordine + fase BRT1 auto
        end
        Sync->>MySQL: upsert OrdineFase (mapping reparto via RepartoService)
        Sync->>Events: dispatch OrdineSincronizzato
    end

    Sync-->>Wrapper: result {ordini_creati, aggiornati, fasi_create}
    Wrapper-->>Cron: log + excel:sync trigger
```

Note operative:

- MES preferisce **ATTDocRighe.Descrizione** (commerciale live) su **OC_Descrizione** (PRD stale, può essere obsoleta dopo revisioni OC cliente)
- Non sovrascrive **data_prevista_consegna** se modificata manualmente nel MES

- Non sovrascrive `cliente_nome` se modificato manualmente
- Fase BRT1 (spedizione) viene auto-aggiunta per ogni nuova commessa con priorità 96
- Comando singola commessa: `php artisan onda:sync 0067339-26` (no filtro data)

File: `app/Modules/Onda/Services/OrdineSyncService.php` , `CommessaSyncService.php` , `Adapters/OndaErpAdapter.php` .

Flusso 2: Ciclo fase operatore (Avvia → Pausa → Termina)

Attori: operatore (tablet), `ProduzioneController` , `FaseTransitionService` .

sequenceDiagram

participant Op as Operatore (tablet)
participant UI as /operatore/dashboard
participant Ctrl as ProduzioneController
participant Svc as FaseTransitionService
participant DB as MySQL
participant Events
participant Listener as PropagaFasiSuccessive

Op->>UI: card commessa stato=1 Pronto
Op->>UI: click Avvia
UI->>Ctrl: POST /produzione/avvia {faseId, operatoreId}
Ctrl->>Svc: transizione(fase, Avviato)
Svc->>Svc: PausaRule.canTransitare()
Svc->>DB: UPDATE ordine_fasi SET stato=2, data_inizio=NOW()
Svc->>Events: FaseAvviata(fase, op)
Events-->>Ctrl: ok
Ctrl-->>UI: json success

Note over Op: ...lavoro in corso...
Op->>UI: input qta_prod + scarti
UI->>Ctrl: POST /produzione/aggiorna-campo
Ctrl->>DB: UPDATE ordine_fasi.qta_prod = X

Op->>UI: click Pausa (motivo: cambio materiale)
UI->>Ctrl: POST /produzione/pausa {motivo}
Ctrl->>Svc: pausa(fase, "cambio materiale")
Svc->>DB: stato='cambio materiale'
Svc->>DB: incrementa pivot secondi_pausa

Op->>UI: click Riprendi
UI->>Ctrl: POST /produzione/riprendi
Ctrl->>Svc: riprendi(fase)
Svc->>DB: stato=2 Avviato

Op->>UI: click Termina
UI->>Ctrl: POST /produzione/termina
Ctrl->>Svc: transizione(fase, Terminato)
Svc->>DB: stato=3, data_fine=NOW()

```
Svc->>Events: FaseTerminata(fase, op)

Events->>Listener: handle FaseTerminata
Listener->>DB: SELECT fasi_successive (config sequenza_fasi)
loop fasi successive
    Listener->>DB: UPDATE stato=1 (Pronto) se prec terminate
    Listener->>DB: ricalcola priorit _m37
end
Listener->>Events: CommessaCompletata se tutte fasi >= 3
```

Stati `OrdineFase.stato` :

- 0 = NonIniziata (caricato)
- 1 = Pronto
- 2 = Avviato
- 3 = Terminato
- 4 = Consegnato
- 5 = Esterna
- stringa = Pausa con motivo

File: `app/Http/Controllers/ProduzioneController.php` ,
`app/Modules/Fasi/Services/FaseTransitionService.php` ,
`app/Modules/Scheduling/Services/PropagazioneService.php` .

Flusso 3: Generazione DDT spedizione

```
sequenceDiagram
    participant Sped as Spedizione
    participant UI as /spedizione/dashboard
    participant Ctrl as DashboardSpedizioneController
    participant Svc as DdtSyncService
    participant PdfSvc as DdtPdfService
    participant BRT as BrtService
    participant DB as MySQL
    participant Notif as NotificaService

    Sped->>UI: vede commessa "Da consegnare" (tutte fasi prod terminate)
    Sped->>UI: click "Consegna Totale"
    UI->>Ctrl: POST /spedizione/invio {faseId, tipo: totale}
    Ctrl->>DB: INSERT ddt_spedizioni (numero auto)
    alt vettore = BRT
        Ctrl->>BRT: prepareSoap XML
        BRT-->>Ctrl: shipment_id, segnacollo
        Ctrl->>DB: UPDATE ddt_spedizioni SET segnacollo_brt=X
    end
    Ctrl->>PdfSvc: genera DDT PDF
    PdfSvc-->>Ctrl: PDF storage path
    Ctrl->>DB: UPDATE ordine_fasi BRT1 SET stato=3
    Ctrl->>Ctrl: genera etichetta DataMatrix
    Note over Ctrl: GTIN "A" + 13 cifre, qta zero-pad 8 cifre
    Ctrl->>Notif: notifyOwner "Commessa X spedita"
    Notif->>UI: chat note consegne (polling 15s)
    Ctrl-->>UI: redirect dashboard + PDF link
```

File: `app/Http/Controllers/DashboardSpedizioneController.php` ,
`app/Modules/Spedizione/Services/DdtSyncService.php` ,
`app/Modules/Documenti/Services/EtichettaGenerator.php` .

Flusso 4: Sync Prinect (stampa offset XL106)

Frequenza: **ogni 5 minuti** (cron `prinect:sync-attivita` , storico 7gg).

sequenceDiagram

participant Cron

participant Svc as PrinectSyncService

participant Adapter as PrinectHttpAdapter

participant API as Prinect REST API

participant AutoTerm as PrinectAutoTerminaService

participant DB as MySQL

participant Events

Cron->>Svc: php artisan prinect:sync-attivita

Svc->>Adapter: getDeviceActivity(XL106, oggi, ora)

Adapter->>API: GET /rest/device/{id}/activity

API-->>Adapter: activities

Adapter-->>Svc: collection

loop per ogni attività

Svc->>Svc: estrai jobId

Svc->>Svc: mapping jobId → commessa (ltrim 7 char)

Svc->>Adapter: getJobWorksteps(jobId)

Adapter->>API: GET /rest/job/{jobId}/workstep

Adapter-->>Svc: worksteps

loop per ogni workstep COMPLETED

alt workstep COMPLETED senza actualStartDate

Note over Svc: Fix bug 66811: auto-termina fase MES

Svc->>DB: UPDATE ordine_fasi SET stato=3, data_fine=NOW()

else

Svc->>AutoTerm: deveTerminare(fase)

alt true (>4h inattiva, fasi succ terminate)

Svc->>DB: UPDATE ordine_fasi SET stato=3

end

Svc->>AutoTerm: deveRipristinare(fase)

alt true (stato=3 con attività recenti)

Svc->>DB: UPDATE ordine_fasi SET stato=2

end

end

Svc->>Adapter: getAccounting(jobId, wsId)

Svc->>DB: UPDATE qta_prod_prinect, scarti_prinect, tempo_avviamento_sec, tempo_ese

Svc->>Events: WorkstepCompleted

```
        end
    end

    Svc-->>Cron: log + count fasi aggiornate
```

File: `app/Modules/Prinect/Services/PrinectJobsService.php` ,
`PrinectAutoTerminaService.php` , `Adapters/PrinectHttpAdapter.php` .

Flusso 5: Scheduler Mossa 37

Trigger: `FaseTerminata` event (real-time) OPPURE cron `scheduler:run` (oggi disabled).

```
sequenceDiagram
    participant Trigger as FaseTerminata event<br/>or cron scheduler:run
    participant Propag as PropagazioneService
    participant Prio as PrioritaService
    participant Rules as Rules (4 livelli)
    participant DB as MySQL

    Trigger->>Propag: propagaDopoTermine(faseTerminata)
    Propag->>DB: SELECT fasi successive stessa commessa<br/>(config/sequenza_fasi)
    loop fasi successive
        Propag->>Rules: SequenzaFasiRule.puoIniziare(fase)
        alt prec tutte terminate
            Propag->>DB: UPDATE disponibile=true
        end
    end

    Propag->>Prio: calcolaPriorita per tutte le fasi disponibili
    loop ogni fase
        Prio->>Rules: livello 1 Disponibilità (peso 1000)
        Rules-->>Prio: 1000 o 0
        Prio->>Rules: livello 2 UrgenzaRule.urgenza()<br/>(giorni residui, negativo=ritardo)
        Rules-->>Prio: float urgenza
        Prio->>Rules: livello 3 BatchAffinityRule<br/>(setup compatibili ±5gg)
        Rules-->>Prio: batch_key + bonus
        Prio->>Rules: livello 4 SequenzaFasi (config)
        Rules-->>Prio: peso sequenza
        Prio->>Prio: somma lessicografica → prioritá_m37
        Prio->>DB: UPDATE prioritá_m37, sequenza_m37
    end

    Note over Prio: Dashboard operatore ora vede fasi ordinate DESC per prioritá_m37
```

Macchine specifiche:

- XL106: 24h lun-ven (3 turni)
- JOH (caldo): 6-22 lun-ven (2 turni)
- BOBST: 1 macchina, 2 config (rilievi/fustelle), cambio config = 1h setup
- Piegaincolla: 1 macchina, 3 config (PI01/PI02/PI03), cambio config = 1h setup
- Standard 6-22: STEL, PLAST, FIN, INDIGO, TAGLIO, LEGAT, ZUND, MGI

Colli bottiglia: JOH > BOBST > Piegaincolla.

File: `app/Modules/Scheduling/Services/PrioritaService.php` , `PropagazioneService.php` ,
`Rules/UrgenzaRule.php` , `BatchAffinityRule.php` .

Flusso 6: Sync Excel bidirezionale

Frequenza: **ogni 2 minuti** (cron `excel:sync`).

```
sequenceDiagram
    participant Cron
    participant Svc as ExcelSyncService
    participant FS as Filesystem
    participant DB as MySQL

    Cron->>Svc: php artisan excel:sync

    Note over Svc: 1. syncIn (Excel → DB)
    Svc->>FS: lettura sync_data.xlsx
    Svc->>Svc: calcola MD5 hash
    Svc->>FS: legge .last_sync_hash
    alt hash diverso
        Svc->>Svc: parsing colonne A-AJ (PhpSpreadsheet)
        Svc->>Svc: raggruppa per commessa
        loop righe modificate
            Svc->>DB: UPDATE OrdineFase corrispondente
            alt data_prevista_consegna cambiata
                Svc->>DB: propaga a tutti ordini stessa commessa
            end
        end
    end
    loop righe eliminate (presenti in .last_exported_ids ma non in.xlsx)
        Svc->>DB: soft-delete OrdineFase
    end
    Svc->>FS: aggiorna .last_sync_hash, .last_sync_timestamp
end

Note over Svc: 2. syncOut (DB → Excel)
Svc->>DB: SELECT current state ordini + fasi
Svc->>FS: scrivi sync_data.xlsx (PhpSpreadsheet)
Svc->>FS: aggiorna .last_exported_ids
```

File: `app/Http/Services/ExcelSyncService.php` (legacy),

`app/Modules/Documenti/Services/ExcelSyncService.php` (nuovo).

State files: `.last_sync_timestamp`, `.last_sync_hash` (MD5), `.last_exported_ids`.

Flusso 7: Login operatore + Login admin con 2FA

7a. Login operatore (codice + password opzionale)

```
sequenceDiagram
    participant Op as Operatore
    participant UI as /operatore/login
    participant Ctrl as OperatoreLoginController
    participant Auth as Auth guard 'operatore'
    participant DB as MySQL

    Op->>UI: codice_operatore + password
    UI->>Ctrl: POST /operatore/login
    Ctrl->>Auth: attempt({codice_operatore, password})
    Auth->>DB: SELECT operatori WHERE codice_operatore=X AND attivo=1
    DB-->>Auth: operatore
    Auth->>Auth: Hash::check(password)
    Auth-->>Ctrl: ok
    Ctrl->>DB: INSERT operatore_tokens (token random, expires +12h)
    Ctrl->>UI: redirect /operatore/dashboard?op_token=XXX
    Note over UI: Per-tab session via op_token query param
```

7b. Login admin con 2FA TOTP

```
sequenceDiagram
    participant Adm as Admin
    participant UI as /admin/login
    participant Login as AdminLoginController
    participant Auth as Auth guard 'web'
    participant TFC as TwoFactorController
    participant TFS as TwoFactorService (Google2FA)
    participant DB

    Adm->>UI: email + password
    UI->>Login: POST /admin/login
    Login->>Auth: attempt({email, password})
    Auth->>DB: SELECT users WHERE email=X
    DB-->>Auth: user
    Auth-->>Login: ok
    Login->>DB: SELECT user.2fa_enabled_at
    alt 2FA abilitato
        Login->>UI: redirect /admin/2fa/challenge
        Adm->>UI: codice TOTP (Google Authenticator)
        UI->>TFC: POST /admin/2fa/verify {code}
        TFC->>TFS: verify(user.2fa_secret, code)
        TFS-->>TFC: ok
        alt trust device requested
            TFC->>DB: INSERT trusted_devices (cookie hash SHA256, expires +10yr)
        end
        TFC->>UI: redirect /admin/dashboard
    else
        Login->>UI: redirect /admin/dashboard
    end
end
```

File: `app/Http/Controllers/OperatoreLoginController.php`, `AdminLoginController.php`, `TwoFactorController.php`, `app/Services/TwoFactorService.php`.

Recovery codes: 8 codici 4-4 one-time use generati al setup 2FA.

Eventi cross-modulo (catalogo completo)

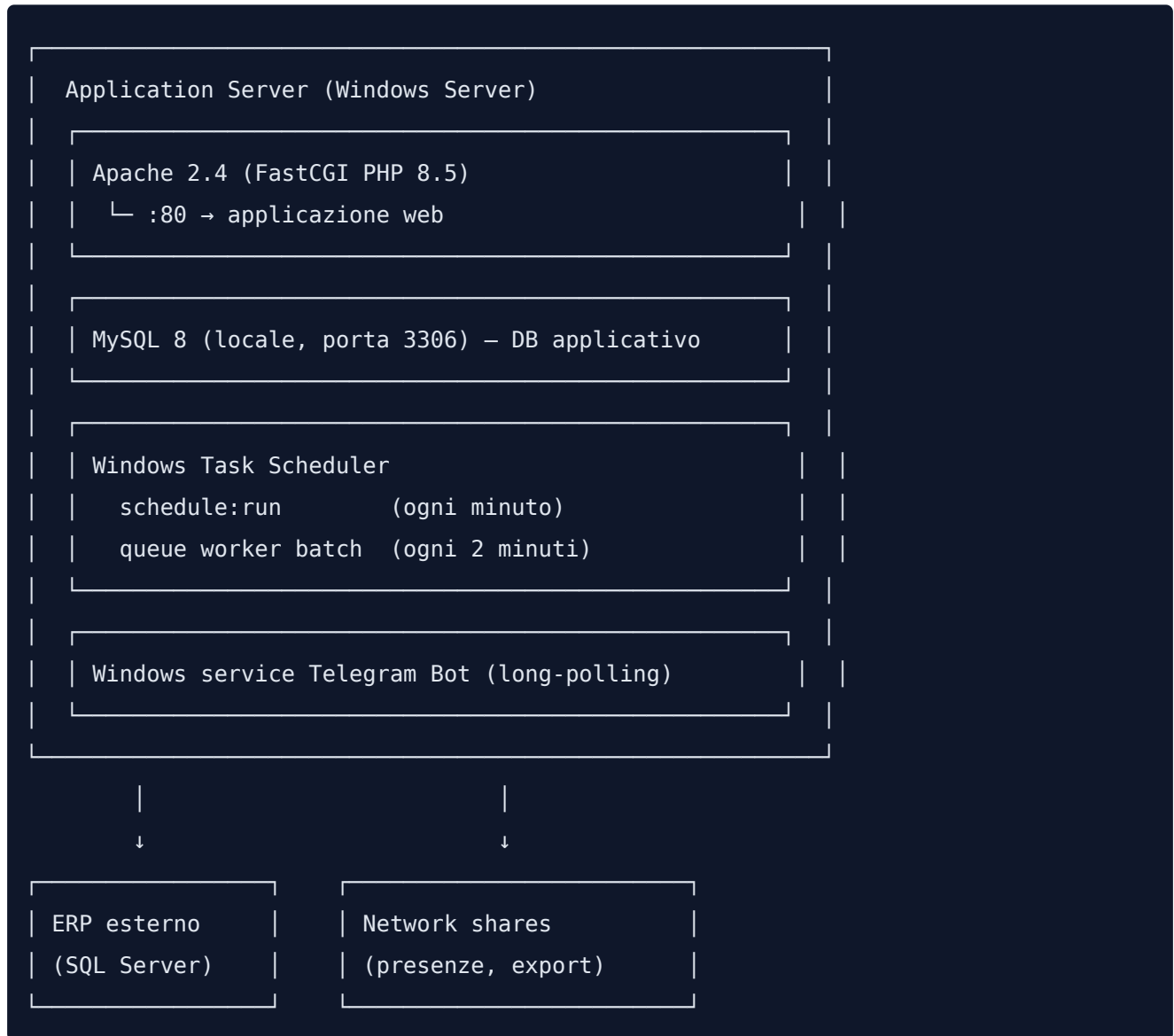
Evento	Modulo origine	Listener
FaseAvviata	Fasi	TracciaInizioFase (Audit)
FaseTerminata	Fasi	PropagaFasiSuccessive (Scheduling), NotificaCommessaCompletata (Commessa)
CommessaCompletata	Commessa	NotificaSpedizione (Notifiche), AuditLog
CommessaConsegnata	Commessa	AuditLog
OrdineSincronizzato	Onda	ExcelSync trigger (Documenti)
ClienteSincronizzato	Onda	AuditLog
WorkstepAvviato	Prinect	AuditLog
WorkstepCompleted	Prinect	UpdateOrdineFase tempi, AuditLog
SottoSogliaEvento	Magazzino	NotificaSottoSoglia (Notifiche)
SpedizioneInRitardo	Spedizione	NotificaSpedizioneInRitardo (Notifiche), EmailCliente
FustellaPrelevata	Fustelle	AuditLog
FustellaRestituita	Fustelle	AuditLog
NotaFustellaAggiunta	Fustelle	NotificaOperatore (Notifiche)
TimbraturaRegistrata	Presenze	AuditLog, CalcoloOreService
OperatoreNonTimbrato	Presenze	NotificaAlert (Notifiche)
StraordinariSuperati	Presenze	NotificaAlert (Notifiche), AuditLog
RepartoCapacitaSuperata	Reparti	NotificaOwner (Notifiche)

Tutti dispatchati via Laravel Events (`event(new EventClass(...))`) e handler in `app/Modules/<Modulo>/Listeners/` .

Registrazione listener: `app/Providers/EventServiceProvider.php` o auto-discovery con `__invoke()` listener.

08. Deployment

Architettura produzione



Stack runtime

Componente	Versione	Note
OS	Windows Server	Tipicamente 2019/2022
Web server	Apache 2.4	PHP via FastCGI
PHP	8.5 (min 8.2)	Sapi: fpm/fastcgi
MySQL	8.0 LTS	Engine InnoDB, utf8mb4
Composer	2.5+	Autoload optimized in prod
Node	18+	Build asset Vite
Git	2.40+	Pull deploy

IIS NON usato: intercetta porta 80 e crea conflitti . Apache su porta 80 dedicata.

Cron / Task Scheduler

Windows Task Scheduler esegue `\LaravelScheduler` come SYSTEM ogni minuto:

```
php artisan schedule:run
```

File: `routes/console.php` (13 task schedulati):

#	Comando	Frequenza	Condizione
1	<code>onda:sync</code>	Ogni ora	24/7
2	<code>excel:sync</code>	Ogni 2 min	24/7
3	<code>fiery:sync</code>	Ogni minuto	24/7
4	<code>princt:sync-attivita</code>	Ogni 5 min	24/7 (storico 7gg)
5	<code>fiery:snapshot-contatori</code>	Feriali 16:55	Pre-shutdown 17:00
6	<code>fiery:export-contatori --mese-corrente --email={REPORT_CONTATORI_T0}</code>	Ultimo del mese 17:00	Dopo snapshot
7	<code>presenze:sync</code>	Ogni minuto	Feriali 05-23
8	<code>presenze:export-excel</code>	Ogni 15 min	Feriali 05-23
9	<code>audit:pulisci</code>	Giornaliero 03:00	Retention 90gg default
10	<code>cliche:match</code>	Ogni 10 min	Dopo Onda sync
11	<code>scheduler:run --email --to={PIANO_EMAIL_T0}</code>	Giornaliero <code>PIANO_EMAIL_ORA</code>	Feriali, <code>PIANO_EMAIL_ENABLED=true</code>
12	<code>ritardi:alert</code>	Feriali 07:30	<code>ALERT_RITARDI_ENABLED=true</code>
13	<code>scheduler:run</code> (Mossa 37 ottimizzatore)	Ogni 15 min	Attivo

Variabili env per cron

Variabile	Default	Uso
<code>REPORT_CONTATORI_T0</code>	<code>admin@cliente.it</code>	Email report contatori V900
<code>PIANO_EMAIL_ENABLED</code>	<code>false</code>	Abilita email piano produzione
<code>PIANO_EMAIL_ORA</code>	<code>06:00</code>	Ora invio piano
<code>PIANO_EMAIL_T0</code>	<code>direzione@cliente.it</code>	Destinatari piano (csv)
<code>ALERT_RITARDI_ENABLED</code>	<code>false</code>	Abilita alert ritardi

Queue + Worker

Voce	Valore
Driver	<code>database</code> (env <code>QUEUE_CONNECTION</code>)
Tabella jobs	<code>jobs</code>
Tabella failed	<code>failed_jobs</code> (driver <code>database-uuids</code>)
Retry after	90s
Worker	<code>queue_worker.bat</code> (PHP artisan queue:work --tries=3 --timeout=300)
Esecuzione worker	Task Scheduler ogni 2 minuti (controllo se già in esecuzione)

Job principali:

- `CachePrinectInkJob` — Caching consumo inchiostro Prinect (long-running)
- `ExcelSyncJob` — Sync bidirezionale Excel (potenzialmente lungo)
- `BollaAIJob` — Analisi foto bolla via Claude Vision (Telegram bot)

Failed jobs management:

```
php artisan queue:failed      # lista
php artisan queue:retry all   # retry tutti
php artisan queue:flush      # pulisci tutti
php artisan queue:retry <uuid> # retry singolo
```

Servizi Windows

Mossa37TelegramBot (nssm)

Long-polling bot Telegram in background:

```
nssm install Mossa37TelegramBot "C:\php\php.exe"
nssm set Mossa37TelegramBot AppParameters "artisan telegram:poll --timeout=30"
nssm set Mossa37TelegramBot AppDirectory "C:pp"
nssm set Mossa37TelegramBot AppRestartDelay 5000
nssm set Mossa37TelegramBot Start SERVICE_AUTO_START
nssm start Mossa37TelegramBot
```

Auto-restart 5s in caso di crash. Log: `storage/logs/laravel.log` .

File system

```
C:pp\  
├─ app/  
├─ bootstrap/  
├─ config/  
├─ database/migrations/  
├─ docs/ # docs interne  
├─ docs/partner/ # docs per partner integratore  
├─ public/ # asset statici  
├─ resources/views/ # blade templates  
├─ routes/web.php, console.php  
├─ storage/  
│   ├─ app/ # PDF DDT, etichette  
│   ├─ app/excel_sync/ # sync_data.xlsx + state  
│   ├─ framework/cache/  
│   ├─ framework/sessions/  
│   └─ logs/laravel.log  
├─ vendor/ # composer dependencies  
└─ .env  
  
C:pp-staging\
```

Network shares

Path	Uso	Credenziali env
\\<IP ERP>\nettime_timbrature\	CSV timbrature NetTime	NETTIME_SHARE_USER , NETTIME_SHARE_PASS
\\<IP gestione presenze>\export\	Export NetTime PC (TODO config)	NETTIME_PC_USER , NETTIME_PC_PASS , NETTIME_EXPORT_CMD

Montaggio share: Windows credential manager o `net use` con `/persistent:yes` .

Environment variables principali (.env)


```
APP_NAME=MES
APP_ENV=production
APP_KEY=base64:...
APP_DEBUG=false
APP_URL=https://app.cliente.it
LOG_LEVEL=error
LOG_CHANNEL=daily
```

```
# DB MES
```

```
DB_CONNECTION=mysql
DB_HOST=db.cliente.local
DB_PORT=3306
DB_DATABASE=mossa37
DB_USERNAME=...
DB_PASSWORD=...
```

```
# DB Onda
```

```
DB_ONDA_HOST=erp.cliente.local
DB_ONDA_PORT=1433
DB_ONDA_DATABASE=...
DB_ONDA_USERNAME=...
DB_ONDA_PASSWORD=...
```

```
# Session/Queue/Cache
```

```
SESSION_DRIVER=database
SESSION_LIFETIME=480
SESSION_ENCRYPT=true
QUEUE_CONNECTION=database
CACHE_STORE=database
```

```
# Prinect
```

```
PRINECT_API_URL=...
PRINECT_USERNAME=...
PRINECT_PASSWORD=...
PRINECT_DEVICE_XL106_ID=4001
```

```
# Fiery
```

```
FIERY_HOST=...
FIERY_USERNAME=...
```

```
FIERY_PASSWORD=...

# BRT
BRT_USERID=...
BRT_PASSWORD=...
BRT_VERIFY_SSL=true

# Telegram
TELEGRAM_BOT_TOKEN=...
TELEGRAM_WEBHOOK_SECRET=...

# Claude Vision (OCR bolle)
ANTHROPIC_API_KEY=...

# Excel
EXCEL_SYNC_PATH=C:pp\storage\app\excel_sync\sync_data.xlsx

# NetTime
NETTIME_SHARE_USER=...
NETTIME_SHARE_PASS=...

# Cron config
REPORT_CONTATORI_T0=admin@cliente.it
PIANO_EMAIL_ENABLED=false
PIANO_EMAIL_ORA=06:00
PIANO_EMAIL_T0=direzione@cliente.it
ALERT_RITARDI_ENABLED=false

# Sicurezza
TRUSTED_PROXIES=*
SESSION_SECURE_COOKIE=true
SESSION_DOMAIN=
FORCE_HTTPS=true
```

Deployment process (manuale)

1. Connessione

- SSH disponibile (oppure AnyDesk/RustDesk per accesso remoto IT)

- `cd C:pp\` (master) o `cd C:pp-staging\` ()

2. Pull + dipendenze

```
git pull origin main          # o
composer install --no-dev --optimize-autoloader
npm ci && npm run build        # se asset frontend modificati
```

3. Migrate + cache

```
php artisan migrate --force
php artisan config:cache
php artisan route:cache
php artisan view:cache
php artisan event:cache      # listener auto-discovery
```

4. Restart

```
# Restart Apache (se config Apache modificato)
net stop Apache2.4 && net start Apache2.4

# Restart queue worker (per ricaricare codice job)
php artisan queue:restart

# Restart Telegram bot service
nssm restart Mossa37TelegramBot
```

5. Verifica

```
curl https://app.cliente.it/health    # → "MES OK"
```

Controllo log:

- `storage/logs/laravel-YYYY-MM-DD.log`
- Apache error log
- Audit log via `/owner/audit-log`

Rollback

```
git reset --hard <previous_commit>
composer install --no-dev
php artisan migrate:rollback          # solo se serve
php artisan config:cache
```

Backup

Cosa	Frequenza	Strumento	Note
MySQL DB	Giornaliero	<code>mysqldump --single-transaction</code>	TODO: documentare cronologia
File storage	Manuale snapshot	TODO automatizzare	
Onda DB	Esterno (fornitore Onda)	gestito da fornitore	
Codice	Git remote (GitHub privato)	Push frequente	

Restore disaster recovery:

1. Restore MySQL dump
2. Clone repo, checkout tag/commit specifico
3. `composer install`
4. Restore `.env`
5. `php artisan migrate --force`
6. Restart services

Monitoring

- **Health endpoint:** `GET /health` → response `"MES OK"` 200
- **Audit log:** tabella `audit_logs`, vista `/owner/audit-log` (90gg default retention)
- **Application log:** `storage/logs/laravel-YYYY-MM-DD.log` (daily channel)
- **Apache log:** `%APACHE_HOME%/logs/`
- **Queue failed:** `php artisan queue:failed`

Per monitoring esterno (uptime, response time) si consiglia health check da servizio cloud su `/health`.

Considerazioni per deploy partner integratore

Multi-tenant: opzioni

Opzione A — Istanza dedicata per cliente (più semplice):

- Codebase clonata per ogni tenant
- DB MySQL separato (`mossa37_clienteX`)
- `.env` per tenant
- Subdomain o porta dedicata
- Cron e queue separati

Opzione B — Multi-tenant column (richiede refactor):

- Tabelle con `tenant_id` column
- Middleware `TenantResolver` per filtrare query
- `TenantManager` per switch DB connection
- Sessioni isolate per tenant

Branch predispone codebase per opzione B (vedi roadmap).

Port mapping consigliato

Porta	Servizio
80/443	Apache MES (HTTPS)
3306	MySQL (non esposto pubblicamente)
1433	SQL Server Onda (se Onda interno)
-	Outbound: Prinect REST, Fiery REST, BRT SOAP, Telegram API, Anthropic API

Requisiti hardware (per ogni tenant)

Risorsa	Minimo	Raccomandato
CPU	4 core	8 core
RAM	8 GB	16 GB
Disco	100 GB SSD	250 GB SSD
Banda	10 Mbps	50 Mbps

Checklist installazione tenant

- ☐ Server Windows o Linux pronto (PHP 8.2+, MySQL 8, Apache 2.4)
- ☐ Repo clonato + composer install
- ☐ DB creato + migrate
- ☐ `.env` configurato con credenziali tenant (Onda, Prinect, Fiery, BRT, ecc.)
- ☐ Cron Task Scheduler configurato (`schedule:run` ogni minuto)
- ☐ Queue worker (`queue_worker.bat` o systemd)
- ☐ Telegram bot service (se richiesto)
- ☐ SSL/HTTPS configurato
- ☐ Backup MySQL automatico
- ☐ Monitoring `/health` esterno
- ☐ Audit log retention configurata
- ☐ Utente admin creato + 2FA attivato
- ☐ Operatori importati con codici + ruoli + reparti
- ☐ Test sync Onda primo run
- ☐ Test sync Prinect/Fiery
- ☐ Smoke test dashboard owner/operatore/spedizione

09. Sicurezza e GDPR

Autenticazione

Dual-guard Laravel

Guard	Provider	Tabella	Accesso
web	users	users	Admin → /admin/*
operatore	operatori	operatori	Staff → /operatore/* , /produzione/* , /spedizione/* , /magazzino/* , /chat/*

Configurazione: `config/auth.php`.

Login operatore

- **Credenziali:** `codice_operatore` (alfanumerico, es. "RB", "MIR") + password opzionale
- **Token per-tab:** `OperatoreToken` (12h expiry) via `op_token` query/header → permette sessioni multiple su tab diversi
- **Fallback:** `session('operatore_id')` se token mancante
- **Throttle:** rate limit `throttle:login` (60 req/min)

Login admin con 2FA TOTP

- **Step 1:** email + password → `AdminLoginController`
- **Step 2:** se `users.2fa_enabled_at` non-null → redirect `/admin/2fa/challenge`
- **Step 3:** codice TOTP (6 cifre da Google Authenticator / Authy)
- **Step 4:** verifica via `pragmarx/google2fa` (PragmaRX)
- **Recovery codes:** 8 codici 4-4 one-time use, generati al setup
- **Trusted devices:** cookie 10 anni SHA256 hash → tabella `trusted_devices`, evita 2FA su device noti

Sessioni

Voce	Valore
Driver	<code>database</code> (env <code>SESSION_DRIVER</code>)
Lifetime	480 minuti (env <code>SESSION_LIFETIME</code>)
Encryption	<code>true</code> (env <code>SESSION_ENCRYPT</code>)
Secure cookie	<code>true</code> in prod (<code>SESSION_SECURE_COOKIE</code>)
Same-site	<code>lax</code> default

CSRF

- **Token refresh** ogni 30min via `/csrf-refresh` (operatore views)
- **Pageshow handler**: ricarica token su back/forward browser
- **CSRF exempt routes** (token-based o webhook):
 - `owner/aggiungi-riga` , `owner/sync-onda` , `owner/import` , `owner/aggiorna-campo` , `owner/cliche/*` , `owner/applica-priorita`
 - `operatore/prestampa/*`
 - `spedizione/sync-onda`
 - `telegram/webhook/*` (HMAC secret)

File auth coinvolti:

- `config/auth.php`
- `app/Http/Controllers/AdminLoginController.php`
- `app/Http/Controllers/OperatoreLoginController.php`
- `app/Http/Controllers/TwoFactorController.php`
- `app/Services/TwoFactorService.php`
- `app/Models/OperatoreToken.php`
- `bootstrap/app.php` (CSRF exceptions, 419 handler)

Middleware sicurezza

Middleware	Path	Scopo
SecurityHeaders	global append	HSTS, CSP, X-Frame-Options, X-Content-Type-Options, Referrer-Policy, Permissions-Policy
GzipResponse	global append	Compressione output (-70% bytes, -1s LCP)
AdminAuth	/admin/*	Auth admin + token fallback
OperatoreAuth	/operatore/* , /produzione/* , /spedizione/* , /chat/*	Auth operatore + token (op_token)
OwnerMiddleware	/owner/*	Check ruolo owner
OwnerOrAdmin	/mes/princt/* , /mes/fiery/* , /report-percorso	Owner OR admin (extras opzionali: fiery_contatori)
MagazzinoAuth	/magazzino/*	Owner/owner_readonly/admin OR reparti spedizione
throttle:login	/operatore/login , /admin/login	Rate limit 60 req/min

Pattern token-first (Operatore/Admin):

1. Cerca op_token in query/header → OperatoreToken::valido() scope
2. Se valido: imposta auth context
3. Fallback: session('operatore_id')
4. Se entrambi mancano:
 - API expect JSON → 401
 - HTML → redirect login

Headers HTTP sicurezza

Tutti impostati da SecurityHeaders middleware:

Header	Valore	Scopo
Strict-Transport-Security	max-age=31536000 (1 anno)	HSTS — solo su HTTPS attivo
Content-Security-Policy-Report-Only	script-src 'self' 'unsafe-inline' 'unsafe-eval' ...	CSP report-only (legacy onclick/Echo)
X-Frame-Options	SAMEORIGIN	Anti-clickjacking
X-Content-Type-Options	nosniff	Anti MIME-sniffing
Referrer-Policy	strict-origin-when-cross-origin	Privacy
Permissions-Policy	camera=(), microphone=(), geolocation=()	Blocca API browser sensibili

Note:

- HSTS attivato solo dopo verifica HTTPS stabile (env `force_https=true`)
- CSP **Report-Only** (non bloccante) per via codice legacy con `unsafe-inline` / `unsafe-eval` → migrazione completa programmata
- Trusted proxies: env `TRUSTED_PROXIES` (default `*` per tunnel tnnl.in, in prod whitelist `<host>,192.168.1.0/24`)

Audit log

Tabella `audit_logs`

```

id          BIGINT PK
user_id     BIGINT NULL  -- non FK (log conservato dopo eliminazione utente)
user_name   VARCHAR(100)
action      VARCHAR(50)
model       VARCHAR(100)
model_id    BIGINT
old_values  JSON
new_values  JSON
ip          VARCHAR(45)
user_agent  VARCHAR(500)
extra       TEXT
created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP

```

Indici: `user_id` , `action` , `model` , `created_at` , composite `(model, model_id)` .

Action types (`TipoAzione` enum)

`Create` , `Read` , `Update` , `Delete` , `Login` , `Logout` , `Export` , `Sync` , `Failed`

Eventi tracciati (`Listeners`)

Listener	Trigger
<code>LogFaseAvviata</code>	<code>FaseAvviata</code> event
<code>LogFaseTerminata</code>	<code>FaseTerminata</code> event
<code>LogCommessaCompletata</code>	<code>CommessaCompletata</code> event
<code>LogMovimentoMagazzino</code>	<code>SottoSogliaEvento</code> , movimento manuale
<code>LogLoginRiuscito</code>	Auth event
<code>LogLoginFallito</code>	Auth failed event

Livelli sicurezza (`LivelloSicurezza` enum)

Livello	Retention	Esempi
<code>Normale</code>	1095 giorni (3 anni)	Fase avviata/terminata, modifiche standard
<code>Sensibile</code>	730 giorni (2 anni)	Login, modifiche ruolo, password reset
<code>Critico</code>	2555 giorni (7 anni)	Eliminazioni, esportazioni dati, accessi audit log

Mask dati sensibili (`DatiSensibiliRule`)

Pre-storage automatico mask:

- `password` → `[REDACTED]`
- `token` , `bearer` → `[REDACTED]`
- `sk-*` (API keys) → `[REDACTED]`
- `api_key` , `secret` → `[REDACTED]`

Applicato in `AuditLogService::log()` su `old_values` / `new_values` / `extra` JSON.

Comando retention

```
php artisan audit:pulisci          # default 90gg
php artisan audit:pulisci --giorni=365
```

Cron giornaliero 03:00 (vedi [08-deployment.md](#)).

File modulo Audit:

- `app/Modules/Audit/Services/AuditLogService.php`
- `app/Modules/Audit/Services/AuditQueryService.php`
- `app/Modules/Audit/Services/ComplianceExportService.php`
- `app/Modules/Audit/Rules/DatiSensibiliRule.php`
- `app/Modules/Audit/Rules/RitenzioneRule.php`
- `app/Modules/Audit/Rules/AccessoLogRule.php`
- `app/Console/Commands/PulisciAuditLog.php`

Compliance GDPR

Art. 15 — Diritto di accesso

```
app(ComplianceExportService::class)->esportaPerOperatore(
    userId: 42,
    from: Carbon::now()->subYear(),
    to: Carbon::now()
);
```

Ritorna array `azione/entita/prima/dopo` per operatore. **Esclude** `user_agent` (rischio fingerprinting). Filtrato + pseudonimizzato.

Art. 17 — Diritto di cancellazione

Non è hard-delete. `PseudonymizationJob` anonimizza:

- `user_name` → `EX-USER-{id}`
- `ip` → ultimo octet azzerato (es. `192.168.1.0`)
- Audit log conservato per retention rule (compliance)

Art. 4 Statuto Lavoratori — Sorveglianza non individuale

```
app(ComplianceExportService::class)->aggregatoSindacale(
    from: Carbon::now()->subMonth(),
    to: Carbon::now()
);
```

Ritorna `[date, action, count]` **senza PII** (no nome operatore, no model_id). Supporta review RSU/RSA collettivo — vietata sorveglianza individuale.

Procedura sindacale obbligatoria

MES traccia produttività individuale (`OrdineFase.operatore_id` + tempi) → **richiesto** accordo RSU/RSA o autorizzazione ITL prima rollout completo (art. 4 L. 300/1970).

Status azienda :

- ☐ Informativa GDPR lavoratori art. 13 firmata da ogni dipendente
- ☐ Accordo RSU/RSA art. 4 — TODO da memoria progetto

Retention automatica (RitenzioneRule)

```
RitenzioneRule::eScaduto(
    createdAt: $log->created_at,
    livello: LivelloSicurezza::Sensibile,
    now: Carbon::now()
);
```

Cron `audit:pulisci` giornaliero 03:00 elimina log scaduti per livello.

Access control audit log (AccessoLogRule)

Ruolo	Letture permesse	Export
<code>operatore</code>	Solo proprio, livello <code>Normale</code>	☐
<code>owner</code>	Tutti, livello <code>Normale</code>	☐
<code>admin</code>	Tutti, tutti livelli (Normale/Sensibile/Critico)	☐
<code>owner_readonly</code>	Tutti <code>Normale</code> , solo lettura	☐

Enforced in `Policy` Laravel (`AuditLogPolicy`).

Best practice applicate

Password

- Hashing: `bcrypt` (default Laravel)
- Cost factor: 12 (Laravel default per PHP 8.5)
- Rotation: nessuna forzata (user-driven)

SQL injection

- **Eloquent ORM** ovunque (bind parameters auto)
- Raw queries (`DB::select(..., [...])`) usano placeholder
- Mai `DB::raw(string user input)` senza sanitizzazione

XSS

- **Blade escape automatico** `{{ $var }}` (htmlspecialchars)
- `{!! $var !!}` (raw) usato solo su contenuto fidato sanitizzato
- Fiery dashboard: `fieryEsc()` + `fieryStatoSafe()` per dati API esterni

File upload

- Whitelist MIME types
- Storage isolato (`storage/app/`), non accessibile diretto
- Path traversal check

Webhook Telegram

- HMAC-SHA256 secret in URL: `/telegram/webhook/{secret}`
- Verifica `hash_equals()` constant-time
- Secret string ≥ 16 caratteri (regex validate `[A-Za-z0-9_-]{16,}`)

Rate limiting

Route	Limit
/operatore/login , /admin/login	60 req/min
API generiche	60 req/min (default Laravel)
Push subscribe	10 req/min per IP

Security audit interno

Data: 2026-05-08

Severity	Count	Status
Critico	1	☐ Da risolvere prima rollout partner
Alto	3	☐
Medio	4	☐☐ Programmato
Basso	3	☐☐

Considerazioni multi-tenant per deploy partner

Isolamento dati

Aspetto	Opzione A (istanza dedicata)	Opzione B (multi-tenant column)
DB	Schema MySQL separato per tenant	Schema condiviso + <code>tenant_id</code> column
Codice	Codebase clonata per tenant	Codebase singola
Sessioni	Naturalmente isolate	Cookie domain isolato
Audit log	Tabella isolata	<code>audit_logs.tenant_id</code>
Backup	Per-tenant	Shared con filtro
Onboarding	Manuale (1-2h per tenant)	Automatizzato (5min)

Raccomandazione per partner

Opzione A per i primi 2-3 tenant (rischio bug multi-tenant basso, controllo dati pieno).

Opzione B dopo refactor completo (sistema attuale predispone).

Checklist sicurezza pre-deploy partner

- ☐ HTTPS forzato (`force_https=true`)
- ☐ HSTS attivo
- ☐ 2FA admin obbligatorio
- ☐ Backup MySQL automatico giornaliero
- ☐ Audit log retention configurata
- ☐ CSP migrato a non-Report-Only (rimozione `unsafe-inline` / `unsafe-eval`)
- ☐ Trusted proxies whitelist (no `*` in prod)
- ☐ `.env` non in git, permessi 600
- ☐ Disaster recovery testato
- ☐ Procedura sindacale art. 4 firmata (per tenant italiani)
- ☐ Informativa GDPR art. 13 firmata da dipendenti tenant
- ☐ DPO nominato (se >250 dipendenti)

